

6 Linearni blok kodovi

6.1 Uvod

U uvodnom izlaganju o blok kodovima je rečeno da se u opštem slučaju blok kod (n, k) može zadati tabelom gde su s jedne strane blokovi od po k informacionih bita – ukupno 2^k mogućih blokova, dok se s druge strane nalaze odgovarajuće kodne reči dužine n , koje su po nekom kriterijumu odabrane od 2^n mogućih “kandidata”. Prema tome, za date vrednosti n i k ima ukupno $\binom{2^n}{2^k}$ mogućih kodova. Naravno, za veće vrednosti n i k tabele

bi zahtevale veliki memorijski prostor, tako da je pogodno da se preslikavanje kombinacija informacionih bita u kodne reči definiše (zada) nekim pravilom. I ovi postupci (algoritmi) mogu biti vrlo različiti, ali postoji jedna posebna klasa blok kodova čiji je matematički opis veoma jednostavan blagodareći aparatu diskretne matematike (čiji su neki elementi ukratko izloženi u Dodatku A). Zahvaljujući takvom matematičkom opisu, moguće je dobiti kodove željenih osobina (minimalno Hemingovo rastojanje itd.), koji u znatnom broju slučajeva omogućavaju i relativno jednostavno dekodovanje, čak i kada se radi o ispravljanju grešaka. Korišćeni matematički aparat takođe omogućava relativno jednostavnu hardversku realizaciju koda i posebno dekodera, što može biti izuzetno važno ako se radi o velikim brzinama prenosa. Treba imati u vidu da je termin “velika brzina prenosa” relativan i da zavisi od postojećeg nivoa razvoja tehnologije. Dok se nekada i brzina od nekoliko megabita u sekundi mogla smatrati velikom, danas su to brzine od nekoliko stotina megabita u sekundi a sutra će to biti brzine od više gigabita, pa i terabita, u sekundi.

Kodovi o kojima je reč u ovome odeljku su **linearni blok kodovi**. Njihova definicija krajnje je jednostavna. Neka je dato polje Galoa $GF(q)$, tj. konačno polje sa q simbola. U polju je definisano sabiranje i množenje elemenata (obe operacije su komutativne). Sekvence od po n elemenata polja čine *vektorski prostor* V dimenzije n nad datim poljem. U vektorskom prostoru je definisano sabiranje vektora, tj. sabiranje odgovarajućih elemenata vektora po zakonu sabiranja iz polja, i množenje vektora skalarom, tj. množenje svih elemenata vektora elementom polja po zakonu množenja iz polja. Vektori iz definisanog prostora čine grupu u odnosu na sabiranje vektora. Takođe se može definisati i skalarni proizvod dva vektora kao zbir proizvoda odgovarajućih elemenata tih vektora (sabiranje i množenje je u polju Galoa!). Definicija linearnog (blok) koda je tada: **Linearni kod je potprostor vektorskog prostora nad $GF(q)$** . U daljem izlaganju biće posmatrano uglavnom binarno polje – $GF(2)$, ali je teorija opšta i važi za linearne kodove sa q različitih simbola, ukoliko se ti simboli dovedu u korespondenciju 1:1 sa simbolima polja $GF(q)$. To znači da postoje linearni blok kodovi ako je broj elemenata u kodnoj listi prost broj ili stepen prostog broja – 2, 3, 4($=2^2$), 5, 7, 8($=2^3$), 9($=3^2$), 11, 13, 16($=2^4$) itd. – posebno je pogodno što za svaki stepen broja 2 postoji odgovarajući linearni kod. Ovo je praktično dovoljno za sve danas aktuelne digitalne sisteme. Naravno da se mogu formirati i kodovi s bilo kojim brojem simbola, definisani na bilo koji način, ali oni ne podležu ovome matematičkom pristupu. Međutim, mogu se i dalje posmatrati vektorski prostori nad konačnim poljima, zadajući pri tome kodne reči na drugi način, a ne kao elemente potprostora. Tada se obično kaže da se posmatraju “nelinearni” kodovi. U ovome odeljku biće izloženi detaljnije elementi koji omogućavaju uvid u strukturu linearnih kodova, ali ne uvek i detaljni algoritmi za projektovanje pojedinih kodova. Često se ovakav način opisivanja blok kodova i zove **algebarska teorija kodovanja** (*algebraic coding theory*).

Prethodna “matematička” definicija linearnog koda kao potprostora vektorskog prostora može se “prevesti” na sledeći način koristeći već uvedene pojmove. Ceo prostor ima 2^n vektora – tačaka – kandidata za kodne reči. Od ovih kandidata treba izabrati 2^k kodnih reči. Ako se ove kodne reči izaberu tako da čine potprostor posmatranog prostora tada je dobijen linearni kod. Pošto po Lagranžovoj teoremi broj elemenata grupe mora biti deljiv brojem elemenata podgrupe, a vektori čine grupu u odnosu na sabiranje, to za svako n i svako k postoji linearni kod (n,k) , jer je 2^n deljivo sa 2^k (takođe je i q^n deljivo sa q^k). Pošto podgrupi mora da pripada neutralni element za sabiranje – vektor svih nula $0(0,0,\dots,0)$, to u linearnom kodu postoji kodna reč $(00\dots 0)$. Za podgrupu mora da važi i *zatvorenost*, pa zbir bilo koje dve kodne reči mora dati kodnu reč. Pošto je Hemingovo rastojanje dva vektora (kodne reči) ravno **Hemingovoj težini** (tj. broju “jedinica”) njihovog zbira, a pošto zbog linearnosti za svaka dva vektora postoji kodni vektor jednak njihovom zbiru, to Hemingove težine kodnih vektora istovremeno predstavljaju i Hemingova rastojanja u kodu. Prema tome, da bi se našlo minimalno Hemingovo rastojanje u kodu dovoljno je naći nenultu kodnu reč najmanje težine. Kodna reč svih nula se može smatrati kao “rastojanje vektora od samog sebe”. Za određivanje verovatnoće greške od interesa je tzv. **raspodela težina** (*weight distribution*), koja se naziva i **spektar kodnih rastojanja** (*distance spectrum*), i koja pokazuje koliko u kodu ima kodnih reči određene težine. Detaljnije o određivanju i korišćenju raspodele težina biće reči kasnije u odeljku 6.2.3.

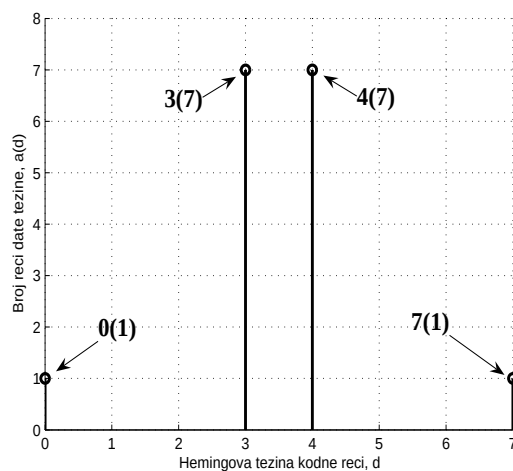
Primeri:

1) Odrediti spektar kodnih rastojanja Hemingovog koda $(7,4)$ (Hemingov kod je linearni kod!).

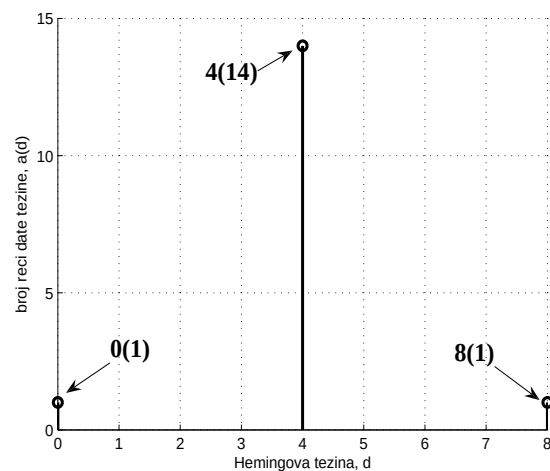
Ovaj kod ima $2^4=16$ kodnih reči, one su:

0000000, 1101001, 0101010, 1000011, 1001100, 0100101, 1100110, 0001111,
1110000, 0011001, 1011010, 0111001, 0111100, 1010101, 0010110, 1111111.

Spektar rastojanja je sledeći: $0(1)$ $3(7)$ $4(7)$ $7(1)$, što znači da postoje po 7 kodnih reči težine 3 i 4 i po jedna reč težine 0 i 7. Na slikama 6.1a i 6.1b prikazani su spektri Hemingovog koda $(7,4)$, odnosno $(8,4)$ dobijenog dodavanjem provere na parnost u prethodnom kodu. Ovaj poslednji spektar se dobija jednostavnom unijom broja kodnih reči s Hemingovim težinama 3 i 4, jer se ovim prvim samo dodaje po jedna “jedinica”, kao i dodavanjem osme “jedinice” najtežoj kodnoj reči.



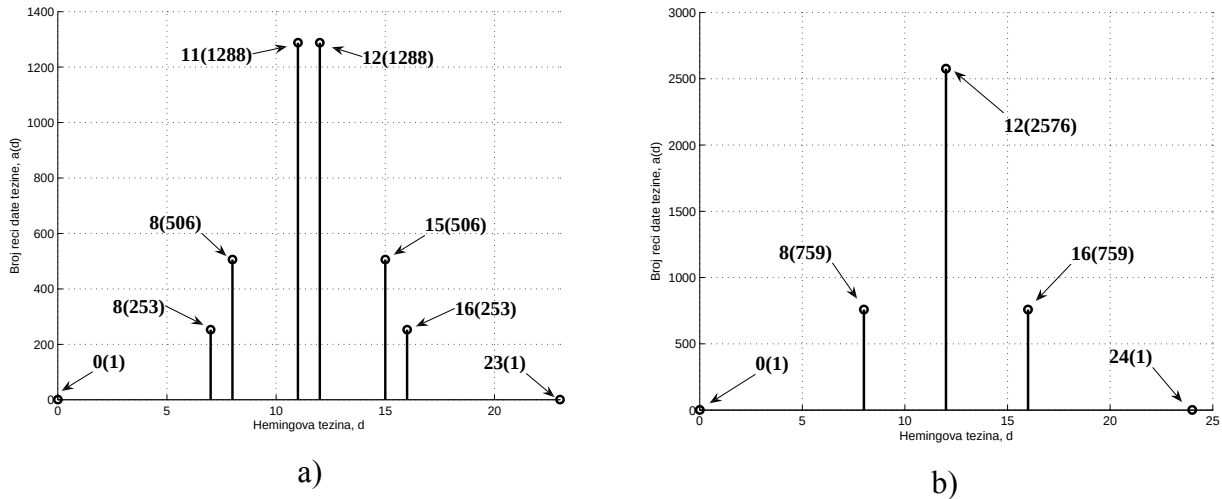
a)



b)

Slika 6.1 Spektar Hemingovog koda $(7,4)$ (a) i Hemingovog koda $(8,4)$ (b)

2) Spektri Golejevog koda (23,12) i proširenog Golejevog koda (24,12) dati su na slikama 6.2a i 6.2b [1]. Lako se vidi da je minimalno rastojanje za kod (23,12) upravo ravno $d=7$, pa kod može da ispravlja sve jednostruke, dvostruke i trostruke greške, dok je ono kod koda (24,12) ravno 8, pa se još mogu i otkrivati četvorostruke greške.



Slika 6.2 Spektar Golejevog koda (23,12) (a) i Golejevog koda (24,12) (b) □

6.1.1 Matrični opis

Poznato je da svaki prostor ima svoju bazu – skup linearno nezavisnih vektora čijim se linearnim kombinacijama (sabiranjem) dobijaju svi vektori prostora. Broj vektora u bazi je ravan dimenziji prostora. Za slučaj koda (n,k) dimenzija celog prostora je n , dok je dimenzija kodnog potprostora k . Prema tome, jedan linearni kod može biti opisan svojom bazom – skupom linearno nezavisnih kodnih vektora koji “generišu” ceo kodni potprostor. Ovi vektori se mogu složiti u obliku matrice dimenzija $k \times n$, te je ova matrica dovoljna da opiše kod, dok bi u opštem slučaju – ako se kod zadaje tabelom – bilo potrebno napraviti tabelu (matricu) dimenzija $2^k \times n$. Matrica se naziva **generišuća matrica** (*generator matrix*) i vrste su joj linearno nezavisne. Njen rang je upravo k . Poznato je da postoje linearne operacije nad vrstama pri kojima se rang matrice ne menja. Ovim operacijama se samo menja baza, ali kod ostaje isti. S druge strane, zamenom odgovarajućih kolona generišuće matrice dobija se generišuća matrica ekvivalentnog koda (ovaj pojam je uveden u petoj glavi).

Primer:

Sledeće kodne reči čine linearni kod $(5,3)$, tj. od $2^5=32$ vektora u prostoru, skup od sledećih $2^3=8$ vektora čini vektorski potprostor:

00000, 10011, 01010, 11001, 00101, 10110, 01111, 11100.

Bilo koje tri linearno nezavisne reči mogu biti izabrane kao baza – za vrste generišuće matrice:

$$\mathbf{G}_1 = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}, \mathbf{G}_2 = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}, \mathbf{G}_3 = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}.$$

Ako se kao prve dve vrste izaberu 10011 i 01010 tada 11001 (njihov zbir) ne može biti treća vrsta, ali se može izabrati bilo koja druga kodna reč, naravno, isključujući 00000. Bilo koja generišuća matrica se može dobiti iz druge elementarnim operacijama nad vrstama. $\square$

Ovakav matematički opis omogućava i jednostavno generisanje kodnih reči na predaji (kodovanje). Neka se posmatra linearni kod (n,k) čija je generišuća matrica (dimenzija $k \times n$)

$$\mathbf{G} = \begin{bmatrix} g_{11} & g_{12} & \cdots & g_{1n} \\ g_{21} & g_{22} & \cdots & g_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ g_{k1} & g_{k2} & \cdots & g_{kn} \end{bmatrix}.$$

Neka treba kodovati vektor informacionih bita $\mathbf{i}(i_1, i_2, \dots, i_k)$. Odgovarajuća kodna reč se može dobiti sledećim množenjem

$$\mathbf{v} = \mathbf{i} \cdot \mathbf{G},$$

gde će rezultujući vektor $\mathbf{v}(v_1, v_2, \dots, v_n)$ biti dobijen kao zbir onih vrsta matrice $\mathbf{G}$ čiji je redni broj ravan pozicijama u informacionom vektoru $\mathbf{i}$ gde se nalazi jedinica. Prema tome, biće sigurno dobijena kodna reč. Takođe, pošto su vrste linearno nezavisne, svakom posebnom informacionom vektoru $\mathbf{i}$ odgovaraće posebna – različita od ostalih – kodna reč $\mathbf{v}$. Ukupan broj ovako dobijenih kodni reči je ravan broju pojedinačnih vrsta matrice $\mathbf{G}$ –

$\binom{k}{1}$, čemu treba dodati broj kombinacija koje nastaju sabiranjem po dve vrste matrice – $\binom{k}{2}$, zatim broj kombinacija nastalih sabiranjem po tri vrste matrice itd. Već je

napomenuto da se sabiraju samo one vrste koje odgovaraju jedinicama u informacionom vektoru, a broj kombinacija jedinica na “dužini” k je upravo $2^k - 1$. Ako se ovome zbiru doda i vektor svih nula (koji odgovara informacionom vektoru svih nula), dobija se

$$\binom{k}{0} + \binom{k}{1} + \cdots + \binom{k}{k} = 2^k.$$

To je istovremeno i broj svih vektora u kodnom potprostoru. Sledeći cilj je dobijanje sistematskog koda. I taj problem je lako rešiv. Elementarnim operacijama nad vrstama, pri čemu se ne menja kod, već samo generišuća matrica, i zamenom mesta kolona, čime se dobija generišuća matrica ekvivalentnog koda, može se ova poslednja napisati u obliku

$$\mathbf{G}_s = \begin{bmatrix} 1 & 0 & \cdots & 0 & p_{11} & \cdots & p_{1,n-k} \\ 0 & 1 & \cdots & 0 & p_{21} & \cdots & p_{2,n-k} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & p_{k1} & \cdots & p_{k,n-k} \end{bmatrix} = [\mathbf{I}_k \quad \mathbf{P}],$$

gde je sa $\mathbf{I}_k$ označena jedinična matrica dimenzija $k \times k$, a $\mathbf{P}$ je matrica dimenzija $k \times (n-k)$. Množenjem informacionog vektora $\mathbf{i}$ matricom ovoga tipa dobija se kodni vektor oblika $\mathbf{v}(i_1, i_2, \dots, i_k, k_1, k_2, \dots, k_{n-k})$. Ovim se dobija sistematski kod jer su informacioni biti u kodnoj reči nepromenjeni, a preostalih $n-k$ bita su provere na parnost dobijene tako da je

$$k_j = i_1 \cdot p_{1j} \oplus i_2 \cdot p_{2j} \oplus \dots \oplus i_k \cdot p_{kj},$$

tj. k_j predstavlja **proveru na parnost** po onim informacionim bitima gde matrica $\mathbf{P}$ u j -toj koloni ima jedinice. Dakle, kolone matrice $\mathbf{P}$ pokazuju pozicije gde se uzimaju provere na parnost. Tih provera ima tačno koliko je potrebno – $n-k$. U binarnom kodu, ovo su stvarno provere na parnost, a u opštem slučaju to su **generalizovane provere na parnost**.

Na ovaj način je rešen problem kodovanja na predaji (tj. kodera). Tu je dovoljno imati matricu $\mathbf{G}$ da bi se izvršilo kodovanje. Postavlja se pitanje šta treba sada raditi na prijemu. Kako proveriti da li je prispeli vektor ($\mathbf{u}$) kodni vektor ($\mathbf{v}$). Svakako da je nepraktično proveravati neposrednom pretragom da li je prispeli vektor linearna kombinacija vektora vrsta generišuće matrice. Aparat diskretne matematike i za dekoader daje elegantno rešenje. U okviru teorije vektorskih prostora postoji i pojam ortogonalnih potprostora. Dva potprostora su **ortogonalni** ako su svi vektori jednog potprostora ortogonalni svim vektorima drugog potprostora, tj. ako su svi skalarni proizvodi ravni nuli. U diskretnoj matematici pokazuje se da svaki potprostor ima sebi ortogonalni potprostor. Da bi se proverila ortogonalnost, dovoljno je proveriti ortogonalnost baza odgovarajućih potprostora, jer ako su vektori jedne baze ortogonalni vektorima iz druge baze, tada će i sve linearne kombinacije vektora jedne baze biti ortogonalne linearnim kombinacijama vektora druge baze. Ako je ukupna dimenzija vektorskog prostora n , takođe se pokazuje da, ako jedan potprostor ima dimenziju k , njemu ortogonalni potprostor ima dimenziju $n-k$. Neka je generišuća matrica (baza) jednog potprostora $\mathbf{G}$ (dimenzija $k \times n$) i neka je $\mathbf{H}$ generišuća matrica (baza) njemu ortogonalnog potprostora, koji mora da ima dimenziju $n-k$. Dimenzije matrice $\mathbf{H}$ su $(n-k) \times n$.

Ako su potprostori ortogonalni, tada mora da važi sledeća osobina (pri množenju matrica svaki element matrice proizvoda a_{ij} se dobija kao skalarni proizvod i -te vrste prve matrice i j -te kolone druge matrice):

$$\mathbf{G} \cdot \mathbf{H}^T = \mathbf{0}.$$

gde $(^T)$ označava transpoziciju a matrica $\mathbf{0}$ ima dimenzije $k \times (n-k)$ i svi su joj elementi ravni nuli. Naravno, imajući u vidu zakone transponovanja, da uz gornje oznake i dimenzije vredi i

$$\mathbf{H} \cdot \mathbf{G}^T = \mathbf{0}^T,$$

Ovo je očigledno, jer su potprostori međusobno ortogonalni. Prema tome, umesto da se na prijemu koristi matrica $\mathbf{G}$, treba rešavajući gornju matričnu jednačinu odrediti **kontrolnu matricu $\mathbf{H}$** (*parity-check matrix*), čije je rang $n-k$, i proveriti da li primljeni vektor pripada ortogonalnom potprostoru za potprostor čija je generišuća matrica (baza) $\mathbf{H}$. Rešavanje matrične jednačine je izuzetno jednostavno ukoliko je matrica $\mathbf{G}$ napisana u obliku koji odgovara sistematskom kodu. Tada se lako pokazuje da je matrica $\mathbf{H}$ oblika

$$\mathbf{H} = \begin{bmatrix} -\mathbf{P}^T & \mathbf{I}_{n-k} \end{bmatrix},$$

gde je $\mathbf{I}_{n-k}$ jedinična matrica dimenzija $(n-k) \times (n-k)$, dok znak “ $-$ ” označava da za svaki element matrice $\mathbf{P}$ treba uzeti njemu suprotni element u odgovarajućem polju. U slučaju binarnog koda – $GF(2)$ – svaki element (0 i 1) je sam sebi suprotan i znak “minus” nije potreban. Očigledno je da su u posmatranom slučaju prvih $n-k$ elemenata svake vrste matrice $\mathbf{H}$ u stvari provere na parnost, dok “jedinica” u jediničnoj matrici ukazuje na poziciju gde se nalazi odgovarajući kontrolni bit. Ukoliko se u matrici $\mathbf{G}$ prvo napiše matrica $\mathbf{P}$ pa jedinična matrica tada i podmatrice u matrici $\mathbf{H}$ takođe menjaju mesta.

Izvesni autori daju matricu sistematskog koda u obliku

$$\mathbf{G}_s = [\mathbf{P} \quad \mathbf{I}_k].$$

Ovde se u kodnoj reči nalaze najpre provere na parnost, a njih slede informacioni biti. U ovom slučaju kontrolna marica ima oblik

$$\mathbf{H}_s = [\mathbf{I}_{n-k} \quad -\mathbf{P}^T],$$

i zadržana je ortogonalnost vrsta ovih matrica.

Zbog uzajamne ortogonalnosti potprostora, pada u oči još jedna osobina. Ako je $\mathbf{G}$ generišuća matrica linearnog koda (n, k) i $\mathbf{H}$ odgovarajuća kontrolna matrica, tada je i $\mathbf{H}$ generišuća matrica linearnog koda $(n, n-k)$ a $\mathbf{G}$ njegova kontrolna matrica. Ovi kodovi se nazivaju **dualni kodovi**.

Jasan je i opšti slučaj određivanja kontrolne matrice iz generišuće. Treba generišuću matricu dovesti na matricu za sistematski kod, odrediti kontrolnu matricu na osnovu gornjeg postupka i eventualno vratiti permutacijama kolone kontrolne matrice na odgovarajuća mesta, ukoliko su pri dovođenju matrice $\mathbf{G}$ u sistematski oblik vršene permutacije kolona. Naravno, i matrica $\mathbf{H}$ se sada, elementarnim operacijama nad vrstama, može menjati po želji.

Na osnovu ortogonalnosti generišuće i kontrolne matrice može se dalje pokazati da jedan blok kod može da ima minimalno Hemingovo rastojanje w ako (i samo ako) je svaki skup od $w-1$ ili manje kolona matrice $\mathbf{H}$ linearno nezavisan. Drugim rečima, da bi se dobio kod s minimalnim Hemingovim rastojanjem w , mora da bude bar $w-1$ linearno nezavisnih provera na parnost [2,3].

Primeri:

1) Za kod (5,3) iz prethodnog primera, matrica $\mathbf{H}$ se lako dobija na osnovu matrice $\mathbf{G}_1$

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Neka je primljeni vektor $\mathbf{v}(v_1, v_2, v_3, v_4, v_5)$. Ako on predstavlja kodnu reč, budući da je V_2 nulti prostor za V_1 , mora da vredi sledeće

$$\mathbf{v}\mathbf{H}^T = \mathbf{0}.$$

Odgovarajuće dimenzije su 1×5 za $\mathbf{v}$, 5×2 za $\mathbf{H}^T$ i 1×2 za $\mathbf{0}$. Ekvivalentne skalarne relacije su

$$\begin{aligned} 1v_1 + 1v_2 + 0v_3 + 1v_4 + 0v_5 &= 0 \\ 1v_1 + 0v_2 + 1v_3 + 0v_4 + 1v_5 &= 0. \end{aligned}$$

Ovo su u stvari provere (kontrole) na parnost, odakle i dolazi naziv matrice $\mathbf{H}$.

U ovome primeru $\mathbf{H}$ je generišuća matrica koda (5,2) koji je dualan razmatranom kodu (5,3). Kodne reči ovoga koda su 00000, 11010, 10101 i 01111. Interesantno je da je reč 01111 element kako V_1 tako i V_2 ("ortogonalna je sama sebi").

2) Za već često pominjani Hemingov kod (7,4) generišuća matrica i kontrolna matrica su

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}. \quad \square$$

6.1.2 Sindrom i standardni raspored

Da bi se izvršilo dekodovanje prispelog vektora $\mathbf{u}$, treba na prijemu odrediti vektor

$$\mathbf{S} = \mathbf{u} \cdot \mathbf{H}^T.$$

Ovaj vektor je dobro poznati **sindrom** i ima dimenzije $1 \times (n-k)$ a elementi su mu u stvari rezultati provera na parnost. Ovo se lako zaključuje na osnovu prethodnog razmatranja o određivanju matrice $\mathbf{H}$, kada je matrica $\mathbf{G}$ data u sistematskom obliku.

Što se tiče otkrivanja i ispravljanja grešaka na prijemu (dekodovanja) na osnovu dobijenog sindroma može se odlučiti da nije bilo greške kada je sindrom $\mathbf{S}$ vektor sa svim nulama (ukupno $n-k$ nula). Ako numerička vrednost sindroma nije nula, tj. sindrom ima bar jednu, ili više jedinica, smatra se da je bilo grešaka pri prenosu. Sada se otvaraju dva puta – ako se greške samo otkrivaju (ARQ procedura) ovo je dovoljna indikacija i traži se ponavljanje kodne reči. Ako se greške i ispravljaju (FEC procedura) treba iz vrednosti sindroma odrediti položaj (ili položaje) greške i izvršiti odgovarajuće ispravke. I ovde korišćeni matematički aparat pruža interesantne mogućnosti. Neka je emitovan kodni vektor $\mathbf{v}$ i neka je primljen vektor $\mathbf{u} = \mathbf{v} + \mathbf{e}$, gde je vektor $\mathbf{e}$ – vektor greške uveden u odeljku 4.4.1, tj. vektor koji ima “jedinice” na pozicijama pojave grešaka, dok su mu ostali elementi ravni nuli. Provera na prijemu daje kao rezultat

$$\mathbf{S} = \mathbf{u} \cdot \mathbf{H}^T = (\mathbf{v} + \mathbf{e}) \cdot \mathbf{H}^T = \mathbf{v} \cdot \mathbf{H}^T + \mathbf{e} \cdot \mathbf{H}^T = \mathbf{0} + \mathbf{e} \cdot \mathbf{H}^T = \mathbf{e} \cdot \mathbf{H}^T.$$

Prema tome, na osnovu rezultata sindroma treba odrediti komponente vektora greške $\mathbf{e}$. Očigledno da ako dva vektora greške imaju isti sindrom, tada se ovaj može pripisati samo jednom od njih. Pošto je manji broj grešaka verovatniji od većeg, to od svih vektora greške koji odgovaraju jednoj vrednosti sindroma treba uzeti onaj koji ima najmanje grešaka (“jedinica”). Ovaj vektor greške treba oduzeti – sabrati po modulu 2 u slučaju binarnog koda – od prispelog vektora $\mathbf{u}$ da bi se ispravile greške i dobio kodni vektor $\mathbf{v}$. Sada se takođe vidi prednost Hemingovih kodova. Kod njih sindrom odmah, pročitao kao broj, pokazuje poziciju pogrešnog bita, dok se u opštem slučaju mora vršiti preračunavanje. Naravno, za nebinarne linearne kodove mora se odrediti (ako se greške ispravljaju) i “težina” nastale greške, o čemu će kasnije biti više reči.

Još bolji uvid u otkrivanje i ispravljanje grešaka pomoću linearnih blok kodova može se steći koristeći i dalje aparat diskretne matematike. Linearni kod (n, k) je potprostor linearnog vektorskog prostora dimenzije n . On je istovremeno podgrupa aditivne grupe. Svaka grupa se može rastaviti po podgrupi. Kod linearnog koda takvo rastavljanje se naziva **standardni raspored** (*standard array*). On se može iskoristiti za korekciju grešaka na sledeći način. Kodne reči predstavljaju podgrupu po kojoj se rastavlja i one se ispišu kao prva vrsta, počinjući od reči koja sadrži sve nule. Kao lider sledeće vrste (klase) bira se vektor koji sadrži sve nule izuzev jedne jedinice. Sabiranjem ovoga vektora s kodnim rečima dobijaju se vektori koji se od kodnih razlikuju samo u jednom bitu na datoj poziciji. Drugim rečima, svi vektori ove vrste imaju Hemingovo rastojanje 1 od kodnih reči. Zatim se kao lider sledeće vrste bira vektor koji opet ima sve nule, izuzev jedne jedinice. Ona naravno mora biti na drugome mestu u odnosu na poziciju u lideru prethodne vrste. Ovaj postupak se nastavlja dok se ne iskoriste svi takvi vektori $((n-1) \times "0", 1 \times "1")$. Na ovaj način su oko svake kodne reči opisane “sfere” koje sadrže sve vektore na Hemingovom rastojanju 1 od te kodne reči. Isti postupak se nastavlja, koristeći kao lidere vektore Hemingove težine ravne 2 itd., dok se ne dobije kompletan standardni raspored. Može se reći da se na ovaj način povećavaju poluprečnici sfera. Dekodovanje se može zamisliti na sledeći način. Kada se primi neki vektor, vrši se provera koristeći kontrolnu matricu da li je to kodna reč. Ako

to nije kodna reč, primljeni vektor se traži u standardnom rasporedu. Kada je nađen, pretpostavlja se da on potiče od kodne reči koja je na početku kolone. Kodna reč se dobija oduzimanjem odgovarajućeg lidera kolone od primljenog vektora (u polju $GF(2)$ oduzimanje i sabiranje daju isti rezultat!). Drugim rečima, ceo postupak prenosa se modeluje dodajući vektor greške

$$u=v+e.$$

Dakle, na prijemu se oduzimaju odgovarajući vektori greške. Vektori greške koji se žele da isprave upravo se biraju kao lideri klasa.

Na prvi pogled, napred izloženi postupak ispravljanja grešaka – pretraga po standardnom rasporedu – izgleda zametan. Međutim, to je samo prividno. Lako se može pokazati da svaka klasa ima jedinstveni sindrom. Neka je primljeni vektor $v+e_i$. Odgovarajući sindrom je

$$(v+e_i)H^T = vH^T + e_iH^T = 0 + e_iH^T.$$

Dalje, može se pokazati da će sindromi koji odgovaraju različitim klasama biti različiti.

Na ovome mestu je pogodno izvršiti malu analizu o sposobnosti linearnih kodova da otkrivaju i ispravljaju greške. Neka se posmatra kod (n,k) . Od ukupnog broja mogućih vektora izabrano je 2^k kodnih reči, dok sindrom ima 2^{n-k} različitih vrednosti. Odmah se uočava da postoje vektori greške koji se ne mogu ni otkriti. To su oni vektori koji su po strukturi ravni kodnim rečima, jer zbog zatvorenosti njihov zbir s kodnim vektorima daje opet kodne vektore, pa će sindrom biti ravan nuli. Takvih vektora greške ima tačno 2^k-1 , gde se od ukupnog broja kodnih reči oduzima kodna reč svih nula – ona mora da pripada linearnom kodu, tj. izostavlja se slučaj kada se greška nije dogodila. Ukoliko se greške samo otkrivaju, broj vektora greške koji se mogu otkriti ravan je broju svih onih vektora koji ne pripadaju kodnom potprostoru (2^n-2^k), jer će zbog zatvorenosti pri sabiranju ovih vektora greške s kodnim vektorima, dobijeni vektor “izaći” iz kodnog potprostora. I, na kraju, broj vektora greške koji se mogu ispraviti ravan je broju različitih vrednosti sindroma, umanjenom za 1, što odgovara sindromu koji ima sve elemente ravne nuli, jer se one mogu jednoznačno povezati sa odgovarajućim vektorima greške. Tih vrednosti ima $2^{n-k}-1$.

Primer:

Kod (5,3) iz prethodnih primera. $H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}$ S

00000	10011	01010	11001	00101	10110	01111	11100	00
00001	10010	01011	11000	00100	10111	01110	11101	01
00010	10001	01000	11011	00111	10100	01101	11110	10
10000	00011	11010	01001	10101	00110	11111	01100	11

Svaki vektor od $2^5=32$ mogućih pojavljuje se samo jedan put (rastavljanje grupe). U drugoj vrsti je kao lider izabran vektor 00001 i kao posledica toga vektor 00100 ne može biti izabran kao sledeći lider jer je $00001+00100=00101$ – kodna reč. Isto vredi i za $00010+01000=01010$. Prema tome, samo po jedan od tih vektora može biti korigovan – izabran kao lider. Samo 3 vektora s po jednom greškom se mogu korigovati, a ne svih 5. Drugačije gledano, u kodu postoje dve kodne reči Hemingove težine 2 – ne mogu se korigovati sve jednostruke greške. S druge strane, ako se greške samo otkrivaju, svaki vektor greške koji ne pripada kodu, tj. nije ravan kodnoj reči, a ima ih 24, može se detektovati jer je podgrupa (kod) zatvorena u odnosu na sabiranje.

Interesantno je pogledati dualni kod (5,2): 00000 11010 10101 01111. Pošto je minimalno Hemingovo rastojanje ravno 3, može se očekivati da se sve pojedinačne bit greške mogu ispraviti. Ovo se vrlo jednostavno vidi iz standardnog rasporeda.

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix},$$

	$\mathbf{S}$
00000 11010 10101 01111	000
00001 11011 10100 01110	101
00010 11000 10111 01101	110
00100 11110 10001 01011	001
01000 10010 11101 00111	010
10000 01010 00101 11111	100

00011 11001 10110 01100	011
00110 11100 10011 01001	111

Za poslednja dva reda, za lidere klasa izabrani su vektori greške koji imaju dva pogrešna bita (od 4 moguća vektora) – 00011+01100=01111, 00110+01001=01111). □

Na taj način, korišćeni matematički aparat je omogućio da se, u slučaju linearnih kodova, veoma lako mogu odrediti neki parametri koda i oceniti njegove performanse. U vezi s linearnim kodovima interesantna je i **Singletonova granica**. Ona daje vezu parametara koda i minimalnog rastojanja u kodu. Za linearni kod (n,k) , za kodnu reč s najmanjom težinom $d_{\min}$ vredi

$$d_{\min} \leq n - k + 1,$$

što je upravo i maksimalno moguće minimalno Hemingovo rastojanje koje se može postići datim kodom. Dokaz je jednostavan. U slučaju da se u informacionom vektoru pojavljuje samo jedna jedinica i ako je ta pozicija obuhvaćena sa svih $n-k$ mogućih provera na parnost, maksimalan broj jedinica u odgovarajućoj kodnoj reči ne može biti veći od broja provera uvećanog za taj informacioni bit. Kodovi kod kojih ova nejednakost prelazi u jednakost zovu se “(razdvojnivi) kodovi s maksimalnim rastojanjem” i često označavaju sa MDS (*Maximum-Distance Separable codes*).

6.1.3 Raspodela težina i Makvilijamsovi identiteti

Ako se standardni raspored koristi za dekodovanje (korekciju grešaka) i ako je p verovatnoća greške na BSC, verovatnoća tačnog dekodovanja je [3]

$$P_{\text{kor.dek}} = n_0(1-p)^n + n_1p(1-p)^{n-1} + \dots,$$

gde je n_i broj lidera klasa težine i ($\sum_i n_i = 2^{n-k}$). Dakle, da bi se dobila maksimalna

verovatnoća tačnog dekodovanja, kao lidere treba izabrati one “vektore greške” koji imaju minimalnu težinu.

Za izračunavanje verovatnoće greške koja se ne može detektovati treba naći ostale težine u kodu. **Raspodela težina (spektar)** koda je skup brojeva a_d koji označavaju broj kodnih reči težine d . Greške neće biti detektovane samo kada se vektor greške poklapa s nekom nenultom kodnom reči. Odgovarajuća verovatnoća je [3]

$$P_e = \sum_{d=1}^n A_d p^d (1-p)^{n-d}.$$

U prethodnoj formuli, imajući u vidu da je $d_{\min}$ minimalna težina, sabiranje počinje u stvari od $d_{\min}$. U slučaju kada se vrši i delimična korekcija, dobijaju se složenije formule.

Raspodela težina se može naći potpunom pretragom. Međutim, ako k nije malo, pretraga može da traje veoma dugo. Kada $n-k$ nije veliko mogu se koristiti **Makvilijamsovi identiteti** (*MacWilliams identities*), koji povezuju raspodelu težina koda $V_1(n, k)$ i njemu dualnog koda $V_2(n, n-k)$ nad $GF(q)$. Neka A_i i B_i označavaju broj vektora težine i u V_1 i V_2 , respektivno. Tada vredi [4]

$$q^k \sum_{i=0}^n B_i z^i = \sum_{j=0}^n A_j (1-z)^j [1+(q-1)z]^{n-j}.$$

Shvatajući A_i i B_i kao koeficijente odgovarajućih polinoma $A(z)$ i $B(z)$, za binarni linearni kod može se pisati [3]

$$A(z) = 2^{-(n-k)} (1+z)^n B\left(\frac{1-z}{1+z}\right), B(z) = 2^{-k} (1+z)^n A\left(\frac{1-z}{1+z}\right).$$

Za MDS kod, tj. za kod koji zadovoljava Singletonovu granicu s jednakošću, postoji eksplicitna formula za raspodelu težina. Ako je minimalno rastojanje u kodu $d_{\min}$, tada je $A_0=1$; $A_i=0$ ($1 \leq i \leq d_{\min}-1$) i, za $i \geq d_{\min}$ [4]

$$A_i = \binom{n}{i} (q-1) \sum_{j=0}^{i-d_{\min}} (-1)^j \binom{i-1}{j} q^{i-d_{\min}-j}.$$

Perfektni kodovi (Hemingov, Golejev) imaju binomijalnu raspodelu težina [2].

Primena ovih identiteta biće prikazana na često korišćenim linearnim blok kodovima (cikličnim, a posebno na BCH i RS kodovima) koji će kasnije biti detaljno opisani.

6.1.4 Produktni kodovi

Do sada, sekvence od po k informacionih simbola bile su posmatrane kao informacioni vektori i $n-k$ provera na parnost su dodavane formirajući odgovarajuću kodnu reč (n, k) koda. Međutim, informacioni simboli se mogu posmatrati i kao matrica koja ima k_1 kolona i k_2 vrsta. U tome slučaju, za svaku vrstu i svaku kolonu može se formirati odgovarajuća kodna reč koristeći čak i različite kodove za kolone (n_2, k_2) i redove (n_1, k_1) prema slici 6.3 (PP označava “proveru proveru”). Odgovarajući kod je $(n_1 n_2, k_1, k_2)$ s minimalnim Hemingovim rastojanjem $d_1 d_2$ – zato se i zove produktni. Provere proveru će biti iste bilo da su dobijene po kao provere vrsta ili provere kolona, ako se koriste obične provere na parnost. U odeljku 5.5.1 razmatrano je samo dodavanje po jedne provere na parnost po vrsti i po koloni.

Ideja se može razviti dalje u više od dve dimenzije [5]. Posmatrajući trodimenzionalnu matricu može se dobiti “kodna reč” dužine $n=n_1n_2n_3$. Ovaj postupak se istovremeno može posmatrati kao pokušaj da se dobiju duže kodne reči a da pri tome koder i dekoder nisu toliko kompleksni kao što bi bili za jedan kod iste dužine (n).

INFORMACIONI SIMBOLI	P R O V E R E
PROVERE	PP

Slika 6.3 Formiranje produktnog koda

6.1.5 Kodovi na bazi matrica Adamara

Kodovi opisani u ovome i sledeća dva odeljka su sada već “klasični” i prikazuju se samo ukratko, bez mnogo detalja.

Adamarova (Hadamard) matrica je dimenzija $n \times n$, a elementi su joj realni brojevi $+1$ i -1 . Ove matrice se definišu na sledeći način (rekurzijom) [1,6]

$$H_1 = [1], \quad H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H_4 = H_2^{\otimes 2} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}, \quad H_{2k} = H_k^{\otimes 2} = \begin{bmatrix} H_k & H_k \\ H_k & -H_k \end{bmatrix},$$

gde $(\cdot)^{\otimes}$ označava Kronekerov proizvod matrica. Ova matrica je simetrična, ima podjednak broj elemenata $+1$ i -1 u svakoj vrsti (koloni) izuzev u prvoj i ima sve vrste (kolone) ortogonalne. Vrste (ili kolone) na taj način predstavljaju diskretne Volšove funkcije – sekvence i koriste se u mobilnim sistemima treće generacije. Polazeći od jedne ovakve matrice može se formirati skup od $2n$ kodnih vektora – vrste Adamarove matrice (ima ih n) i iste vrste pomnožene sa -1 . Kodne reči se dobijaju daljom zamenom 1 sa 0 i -1 sa 1 . Minimalno rastojanje u kodu je $n/2$. Broj kodni reči nije veliki. Takođe, ne mora se uvek dobiti linearni kod.

6.1.6 Rid-Malerovi kodovi

Rid-Malerovi (*Reed-Muller*) kodovi [7,8] (RM) su nekada korišćeni i za kosmičke komunikacije. Mariner je 1972. godine jednim ovakvim kodom slao fotografije Marsa. U ovome odeljku se prikazuju imajući u vidu pre svega njihov jednostavan opis i dekodovanje koje se zasniva na nekoj vrsti majoritetne logike. To su linearni kodovi nad $GF(2)$. Za svaki prirodni broj m i prirodni broj r (manji od m) postoji RM kod s dužinom kodne reči $n=2^m$ i $d_{\min} = 2^{m-r}$. To je RM kod r -tog reda (dužine 2^m).

Za ovaj (n,k) kod takođe vredi sledeće

$$k = 1 + \binom{m}{1} + \dots + \binom{m}{r},$$

$$n - k = 1 + \binom{m}{1} + \dots + \binom{m}{m-r-1}.$$

Za objašnjenje konstrukcije definiše se “proizvod” dva vektora na sledeći način:

$$\begin{aligned} \mathbf{a} &= (a_1, a_2, \dots, a_n) \\ \mathbf{b} &= (b_1, b_2, \dots, b_n) \\ \mathbf{ab} &= (a_1 b_1, a_2 b_2, \dots, a_n b_n). \end{aligned}$$

Generišuća matrica ima oblik

$$\mathbf{G} = \begin{bmatrix} \mathbf{G}_0 \\ \mathbf{G}_1 \\ \vdots \\ \mathbf{G}_r \end{bmatrix},$$

gde je prva podmatrica ($\mathbf{G}_0$) vektor dužine 2^m koji sadrži sve jedinice. U sledećoj podmatrici ($\mathbf{G}_1$) dimenzija $m \times 2^m$ se svaka binarna m -torka pojavljuje jedanput kao kolona. U sledećoj ($\mathbf{G}_2$) su vrste proizvodi po dve vrste prethodne matrice (ali se indeksi uzimaju samo u rastućem redosledu), zatim slede proizvodi po tri vrste matrice $\mathbf{G}_1$ itd. Ovo će biti ilustrovano u sledećem primeru.

Primer:

$m=4, n=2^m=2^4=16, r=3$, kod (16,11)

$$\mathbf{G}_0 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = [\mathbf{a}_0],$$

$$\mathbf{G}_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \\ \mathbf{a}_4 \end{bmatrix},$$

$$\mathbf{G}_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 \mathbf{a}_2 \\ \mathbf{a}_1 \mathbf{a}_3 \\ \mathbf{a}_1 \mathbf{a}_4 \\ \mathbf{a}_2 \mathbf{a}_3 \\ \mathbf{a}_2 \mathbf{a}_4 \\ \mathbf{a}_3 \mathbf{a}_4 \end{bmatrix},$$

$$\mathbf{G}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 \mathbf{a}_2 \mathbf{a}_3 \\ \mathbf{a}_1 \mathbf{a}_2 \mathbf{a}_4 \\ \mathbf{a}_1 \mathbf{a}_3 \mathbf{a}_4 \\ \mathbf{a}_2 \mathbf{a}_3 \mathbf{a}_4 \end{bmatrix}.$$

Na taj način generišuća matrica ima dimenzije 15×16 a kod ima minimalno Hemingovo rastojanje $d_{\min} = 2^{m-r} = 2^{4-3} = 2$.

Ako se izaberu samo prve tri matrice ($r=2$) dobija se kod (16,11) s minimalnim Hemingovim rastojanjem $d_{\min} = 2^{m-r} = 2^{4-2} = 4$, čije će dekodovanje sada biti ukratko objašnjeno.

Biti v_i kodne reči su (sa i_j označeni su informacioni biti):

$$\begin{aligned} v_1 &= i_1; v_2 = i_1 \oplus i_5; v_3 = i_1 \oplus i_4; v_4 = i_1 \oplus i_4 \oplus i_5 \oplus i_{11}; v_5 = i_1 \oplus i_3; v_6 = i_1 \oplus i_3 \oplus i_5 \oplus i_{10}; \\ v_7 &= i_1 \oplus i_3 \oplus i_4 \oplus i_9; v_8 = i_1 \oplus i_3 \oplus i_4 \oplus i_5 \oplus i_9 \oplus i_{10} \oplus i_{11}; v_9 = i_1 \oplus i_2; v_{10} = i_1 \oplus i_2 \oplus i_5 \oplus i_8; \\ v_{11} &= i_1 \oplus i_2 \oplus i_4 \oplus i_7; v_{12} = i_1 \oplus i_2 \oplus i_4 \oplus i_5 \oplus i_7 \oplus i_8 \oplus i_{11}; v_{13} = i_1 \oplus i_2 \oplus i_3 \oplus i_6; \\ v_{14} &= i_1 \oplus i_2 \oplus i_3 \oplus i_5 \oplus i_6 \oplus i_8 \oplus i_{10}; v_{15} = i_1 \oplus i_2 \oplus i_3 \oplus i_4 \oplus i_6 \oplus i_7 \oplus i_9; \\ v_{16} &= i_1 \oplus i_2 \oplus i_3 \oplus i_4 \oplus i_5 \oplus i_6 \oplus i_7 \oplus i_8 \oplus i_9 \oplus i_{10} \oplus i_{11}. \end{aligned}$$

Neka je primljena reč $\mathbf{u} = \mathbf{v} \oplus \mathbf{e}$ ($\mathbf{e}$ je vektor greške).

Dekoduju se prvo informacioni biti određeni matricom $\mathbf{G}_2$. Počinje se od poslednjeg informacionog bita i formiraju se četiri sume "ortogonalne" na bit i_{11} :

$$\begin{aligned} S_1 &= u_1 \oplus u_2 \oplus u_3 \oplus u_4 = i_1 \oplus e_1 \oplus i_1 \oplus i_5 \oplus e_2 \oplus i_1 \oplus i_4 \oplus e_3 \oplus i_1 \oplus i_4 \oplus i_5 \oplus i_{11} \oplus e_4 = \\ &= e_1 \oplus e_2 \oplus e_3 \oplus e_4 \oplus i_{11} \\ S_2 &= u_5 \oplus u_6 \oplus u_7 \oplus u_8 = \dots = e_5 \oplus e_6 \oplus e_7 \oplus e_8 \oplus i_{11} \\ S_3 &= u_9 \oplus u_{10} \oplus u_{11} \oplus u_{12} = \dots = e_9 \oplus e_{10} \oplus e_{11} \oplus e_{12} \oplus i_{11} \\ S_4 &= u_{13} \oplus u_{14} \oplus u_{15} \oplus u_{16} = \dots = e_{13} \oplus e_{14} \oplus e_{15} \oplus e_{16} \oplus i_{11}. \end{aligned}$$

Vidi se da svaka od suma zavisi samo od jedanaestog informacionog bita i eventualnih grešaka na kanalu, na šta se i mislilo kada je napred uveden pojam ortogonalnosti. Dekodovanje se vrši korišćenjem majoritetne logike (videti i odeljak 5.6.3 gde se objašnjava dekodovanje konvolucionih kodova majoritetnom logikom). Zatim se formiraju sume ortogonalne na i_{10} , i_9 , i_8 , i_7 , i_6 i za svaki bit uradi takođe dekodovanje majoritetnom logikom.

Na taj način se dekoduju poslednjih 6 informacionih bita. Pretpostavi se da je dekodovanje urađeno ispravno i eliminiše se uticaj tih bita na kodne reči na sledeći način:

$$\mathbf{u}' = \mathbf{r} \oplus [i_6 \ i_7 \ i_8 \ i_9 \ i_{10} \ i_{11}] \otimes \mathbf{G}_2.$$

Prema tome, $\mathbf{u}'$ je kodna sekvenca oslobođena uticaja poslednjih 6 informacionih bita, pod pretpostavkom da su svi ispravno dekodovani. Sada se prelazi na formiranje kontrolnih suma za informacione bite $i_2 - i_5$. Svaki od tih bita učestvuje u formiranju ukupno 8 kodnih bita, pa se formira po 8 kontrolnih suma ortogonalnih na te bite – za bit broj 5 sume su $S_1 = u'_1 \oplus u'_2$, $S_2 = u'_3 \oplus u'_4$, $S_3 = u'_5 \oplus u'_6$, $S_4 = r'_7 \oplus r'_8$, $S_5 = r'_9 \oplus r'_{10}$, $S_6 = r'_{11} \oplus r'_{12}$, $S_7 = r'_{13} \oplus r'_{14}$, $S_8 = r'_{15} \oplus r'_{16}$, za ostale bite analogno se određuju kontrolne sume. Većinska logika se u ovome slučaju radi na 8 bita. Konačno, formira se i kodna sekvenca oslobođena uticaja informacionih bita 2, 3, 4, 5, i 6:

$$\mathbf{u}'' = \mathbf{u}' \oplus [i_2 \ i_3 \ i_4 \ i_5] \otimes \mathbf{G}_1 = [i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1 \ i_1].$$

Većinskom logikom se na osnovu 16 bita sekvence $\mathbf{u}''$ odredi informacioni bit i_1 .

Konačno, interesantno je primetiti da se na osnovu opisanog postupka informacioni biti određuju na osnovu većinskog odlučivanja i po principu "bit po bit", odnosno ne dekoduju se svi biti odjednom. $\square$

6.1.7 Kodovi s malom gustinom provera na parnost

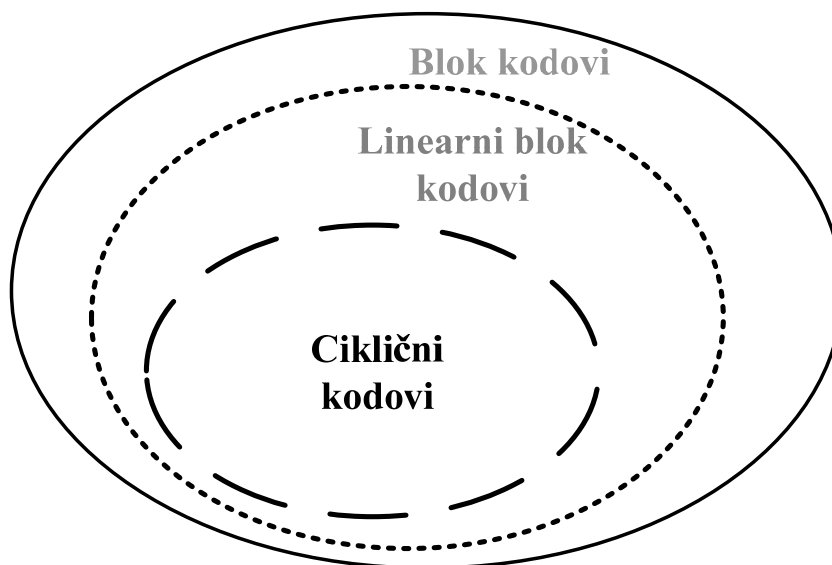
Kodove s malom gustinom provera na parnost (LDPC od “*Low-Density Parity-Check codes*”) pronašao je Galager šezdesetih godina prošloga veka [4]. Kontrolna matrica ovih (n,k) kodova ima K jedinica u svakoj vrsti i J jedinica u svakoj koloni. Pošto mora da vredi $K(n-k)=Jn$, to je kodni količnik dat sa

$$R = 1 - \frac{J}{K}.$$

Vrednosti J i K se obično uzimaju da budu relativno male, često manje od $0,02n$, tako da ovi kodovi imaju zaista malu gustinu provera na parnost. Njihovo dekodovanje se takođe zasniva na korišćenju majoritetne logike. Galager je predložio menjanje primljenih bita koji se nalaze u više provera na parnost koje ne zadovoljavaju, zatim se ove provere ponovo računaju i proces nastavlja dok sve provere ne budu zadovoljene. Ovaj postupak je u suštini iterativan. Interesantno da je odmah predložio i kompleksniji postupak iterativnog dekodovanja koji se zasniva na mekom odlučivanju i aposteriornim verovatnoćama. Može se slobodno reći da je Galager jedan od onih nekoliko istraživača koji su zamislili takvu tehniku kodovanja i dekodovanja čiji su principi veoma bliski onima kod turbo kodovanja. Iterativno dekodovanje LDPC kodova je klasičan primer za ilustraciju iterativne obrade signala [9]. Ovi kodovi će znatno detaljnije biti obrađeni u odeljku posvećenom iterativnom dekodovanju.

6.2 Ciklični kodovi

Iako su ciklični kodovi potklase linearnih, oni ipak zaslužuju poseban veći odeljak imajući u vidu njihovu važnost u savremenim telekomunikacionim sistemima. Na slici 6.4 je prikazan šematski odnos cikličnih kodova u odnosu na blok kodove i njihovu potklasu – linearne blok kodove.



Slika 6.4 Ciklični kodovi kao potklasa linearnih blok kodova

Linearni blok kodovi koriste provere na parnost, ali je njihov opis elegantniji i kompaktniji uz korišćenje aparata diskretne matematike. U okviru ovoga odeljka biće prikazana jedna potklasa linearnih kodova – ciklični kodovi – koji se definišu na još elegantniji način, predstavljajući kodne reči kao polinome s koeficijentima iz konačnog polja. Ciklični kodovi spadaju u onu klasu linearnih kodova koja se veoma često koristi kako samo za otkrivanje, tako i za ispravljanje grešaka. Razlog za to je njihova jednostavna hardverska implementacija, posebno u slučajevima otkrivanja grešaka i ispravljanja jednostrukih grešaka.

Da bi se lakše shvatili ciklični kodovi treba pored pojma vektora uvesti i pojam polinoma. Pri tome se biti (simboli) u kodnoj reči (vektoru) mogu posmatrati kao koeficijenti polinoma po proizvoljnoj promenljivoj x . Kao što su elementi u vektoru uređeni po svojim pozicijama, tako su sada ti elementi uređeni stepenima promenljive čiji su koeficijenti. Vektoru

$$v(a_0, a_1, \dots, a_{n-1})$$

odgovara polinom (s tim što se koeficijenti, pošto se radi o polinomu, mogu pisati u proizvoljnom redosledu)

$$v(x) = a_{n-1}x^{n-1} + \dots + a_1x + a_0.$$

Tako se vektor (110010) može napisati kao polinom $x^5 + x^4 + x + 1$.

Dakle, sada se posmatra skup polinoma s koeficijentima iz nekog konačnog polja. Polinomi se mogu množiti konstantom (elementom polja!), sabirati i međusobno množiti. Sabiranje polinoma je jednostavno – samo se sabiraju koeficijenti uz odgovarajuće stepene promenljive (opet po zakonima iz polja). Pošto se koeficijenti proizvoda polinoma dobijaju kao zbir proizvoda koeficijenata uz odgovarajuće stepene promenljive, to će i novodobijeni polinom imati koeficijente iz istog polja, jer je polje zatvoreno u odnosu na operacije sabiranja i množenja. Međutim, kod množenja polinoma može ipak doći do problema. Pošto je najviši stepen proizvoda dva polinoma ravan zbiru njihovih najviših stepena to rezultat može biti stepena većeg od $n-1$. Sada bi se sa ekvivalentnim vektorima prešlo u drugi vektorski prostor većih dimenzija. Da bi se ovo izbeglo može se pribeći sledećoj proceduri. Kao što se posmatranjem skupa celih brojeva po modulu nekog celog broja m dobijaju ostaci (tj. klase ostataka) u rasponu od 0 do $m-1$, tako se i posmatranjem polinoma po modulu nekog polinoma stepena n dobijaju kao ostaci (klase ostataka) polinomi čiji stepen ne prelazi vrednost $n-1$. Sada se, za razliku od modularne aritmetike, tj. posmatranja celih brojeva po modulu m , gde je bilo m različitih ostataka, kao ostaci javljaju svi polinomi stepena manjeg od n , a tih polinoma ima, u binarnom slučaju – nad $GF(2)$ – upravo 2^n , koliko ima i vektora u posmatranom prostoru. U opštem slučaju, s koeficijentima iz $GF(q)$, ovih polinoma ima q^n . Na ovaj način je definisana algebra polinoma nad poljem koeficijenata. Ukoliko se čitalac na ovome mestu pita zašto je potrebno uvesti množenje polinoma, odgovor je da linearna prekidačka kola upravo mogu da množe i dele polinome, te ako se kodne reči shvate kao polinomi, hardverska realizacija kodera (a često i dekodera) može da bude veoma jednostavna.

Sada je došlo vreme da se definiše ciklični kod. Neka se posmatra vektorski prostor dimenzije n – to je ujedno i dužina kodnih vektora. Potprostor ovoga prostora se naziva **ciklični (kod)** ako za bilo koji vektor

$$v(a_0, a_1, \dots, a_{n-1})$$

vektor

$$v_1(a_{n-1}, a_0, a_1, \dots, a_{n-2})$$

dobijen cikličnom permutacijom njegovih koeficijenata takođe pripada kodnom potprostoru.

Ciklični potprostori su sa svoje strane samo podskup svih mogućih potprostora, pa su ciklični kodovi podskup linearnih kodova. Definicija je jednostavna, ali se postavlja pitanje kako naći ciklične potprostore. Međutim, tu diskretna matematika opet daje jednostavan odgovor. Neka se posmatra algebra polinoma po modulu polinoma x^n-1 nad nekim poljem. Može se pokazati da je jedan potprostor cikličan ako ga čine svi polinomi deljivi nekim polinomom $g(x)$ koji je istovremeno delitelj polinoma x^n-1 (matematički rečeno to je *ideal* u ovoj algebri). Neka je $g(x)$ stepena $n-k$, tada su to linearno nezavisne klase ostataka – polinomi

$$0, g(x), xg(x), \dots, x^{k-1}g(x).$$

Pošto je $g(x)$ stepena $n-k$, to će polinom $x^k g(x)$ biti n -tog stepena, pa će biti predstavljen svojim ostatkom po izvršenom deljenju sa x^n-1 . Ti polinomi, odnosno, odgovarajući vektori, i njihove linearne kombinacije čine ciklični kod. Množenju sa x upravo odgovara cikličan pomeraj koeficijenata polinoma jer je

$$x(a_0 + a_1x + \dots + a_{n-1}x^{n-1}) = a_{n-1} + a_0x + \dots + a_{n-2}x^{n-1}$$

imajući u vidu da je $x^n=1$. Definicija cikličnog koda u matematičkom smislu je sledeća: ***ciklični kod je ideal u algebri polinoma po modulu x^n-1 nad poljem koeficijenata.*** Polinom $g(x)$ je ***generišući polinom*** koda. Bilo koji polinom koji je delitelj x^n-1 generiše svoj kod. Klasa ostataka (može se posmatrati i kao polinom i kao vektor) pripada kodu samo ako je deljiva sa $g(x)$. Ako je

$$x^n-1 = g(x)h(x)$$

tada i $h(x)$ generiše svoj ideal – dualni kod $(n, n-k)$. Polinom $h(x)$ se može nazvati ***kontrolni polinom***. Ako je $g(x)$ stepena $n-k$, tada je dimenzija potprostora k , polinom $h(x)$ je stepena k , i dobija se kod (n, k) .

Sada se postavlja pitanje koji su svi mogući ciklični kodovi pod datim uslovima. Da bi se dobio odgovor na dato pitanje treba polinom x^n-1 razložiti na nesvodljive polinome (tj. polinome koji se dalje ne mogu faktorizovati u datom polju – nemaju u njemu nule). Tada svaki od tih polinoma ima svoj ideal, ali i sve moguće kombinacije proizvoda ovih polinoma daju svoje ideale. To su ujedno i svi mogući ciklični kodovi koji postoje pod datim uslovima. Ako je osnovno polje binarno, tada treba imati na umu da je x^n-1 isto što i x^n+1 , jer je 1 sam sebi inverzni element za sabiranje u datom polju.

Pre razmatranja ilustrativnog primera treba još dovesti u vezu ortogonalnost vektora i “ortogonalnost” polinoma. Pod ovim poslednjim se podrazumeva sledeći izraz

$$g(x) \cdot h(x) = x^n - 1 = 0 \pmod{(x^n - 1)}.$$

Ovaj izraz treba shvatiti tako da x^n-1 podeljen samim sobom daje kao ostatak nulu (polinom!). Neka je potrebno pomnožiti polinom

$$a(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$$

polinomom

$$b(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}$$

i neka je rezultat polinom

$$c(x) = a(x) \cdot b(x) = c_0 + c_1x + \dots + c_{n-1}x^{n-1}.$$

Pri ovome treba imati u vidu da se sve posmatra po modulu polinoma x^n-1 , te da najstariji stepen proizvoda ne može biti veći od $n-1$. Lako se dolazi do zaključka da je odgovarajući koeficijent c_j dat sledećim izrazom

$$c_j = a_0b_j + a_1b_{j-1} + \dots + a_jb_0 + a_{j+1}b_{n-1} + a_{j+2}b_{n-2} + \dots + a_{n-1}b_{j+1}.$$

Dakle, koeficijent c_j dobija se kao zbir proizvoda koeficijenata polinoma koji se množe. Prvi deo rezultata je zbir proizvoda koeficijenata čije je zbir indeksa ravan j , dok je drugi deo zbir proizvoda koeficijenata čiji je zbir indeksa ravan $n+j$. Ovo poslednje je posledica činjenice da je $x^n = x^0 = 1$. Da bi rezultatni polinom $c(x)$ bio ravan nuli, treba da svi njegovi

koeficijenti ($j=0,1,\dots,n-1$) budu ravni nuli. Ako se pažljivije razmotri prethodni izraz, vidi se da je on ravan skalarnom proizvodu vektora $\mathbf{a}$ i vektora dobijenog od vektora $\mathbf{b}$ tako što su mu elementi uzeti u obrnutom redosledu i ciklično permutovani j puta. Dakle, ako je proizvod dva polinoma ravan nuli, tada je vektor koeficijenata jednog polinoma ortogonalan vektoru koeficijenata drugog polinoma uzetih u obrnutom redosledu i svim njihovim cikličnim permutacijama. Pošto je množenje polinoma komutativno, svedeno je koji vektor koeficijenata treba zadržati u normalnom redosledu, a koji uzeti u obrnutom redosledu. U Dodatku A posvećeno je više pažnje vezi vektora i polinoma (odeljak A.7.5).

Izuzetna jednostavnost opisivanja cikličnih kodova može se shvatiti i na osnovu sledeće kratke analize:

- Za kompletan opis (n,k) blok koda potrebno je poznavanje svih 2^k kodnih reči dužine n .
- Za kompletan opis linearnog blok koda sa istim parametrima potrebno je poznavanje njegove baze – k kodnih reči dužine n (generišuća matrica).
- Za kompletan opis odgovarajućeg cikličnog koda dovoljno je znati jednu jedinu kodnu reč dužine n ! Sve ostale kodne reči cikličnog koda dobijaju se kao ciklični pomeraji te kodne reči i kao linearne kombinacije ovih permutacija.

Treba istaći da su ciklični kodovi prilično malobrojni u odnosu na linearne blok kodove istih dimenzija. Npr. za $(n,k)=(7,3)$ postoji $1.34 \cdot 10^{12}$ blok kodova, 11811 linearnih blok kodova, ali su samo dva od njih ciklični!

Postoji više vrsta **trivijalnih cikličnih kodova**, kao što su kodovi zasnovan na ponavljanju, što u binarnom slučaju odgovara kodu $(n,1)$ čije jedine dve kodne reči čine nizovi od n nula i n jedinica. Takođe je i kod (n,n) cikličan. Od binarnih trivijalnih kodova mogu se još pomenuti i kodovi koji se sastoje od svih mogućih kodnih reči s parnim brojem jedinica, kao su kod $(3,2)$ s kodnim rečima (000), (011), (101) i (110) i kod $(4,3)$ s kodnim rečima (0000), (0011), (0101), (0110), (1001), (1010), (1100) i (1111). Interesantno je da su za neke dužine kodne reči, npr. za $n=19$, binarni trivijalni ciklični kodovi istovremeno i jedini postojeći ciklični kodovi.

Primeri:

1) Neka je dat polinom x^7+1 nad poljem GF(2). Ovde je

$$x^7+1 = (1+x)(1+x+x^3)(1+x^2+x^3).$$

Neka je generišući polinom

$$g(x) = 1+x^2+x^3$$

Na ovaj način se dobija kod $(7,4)$ jer je $n=7$, $n-k=3$, a ciklični kod ima dimenzije (n,k) .

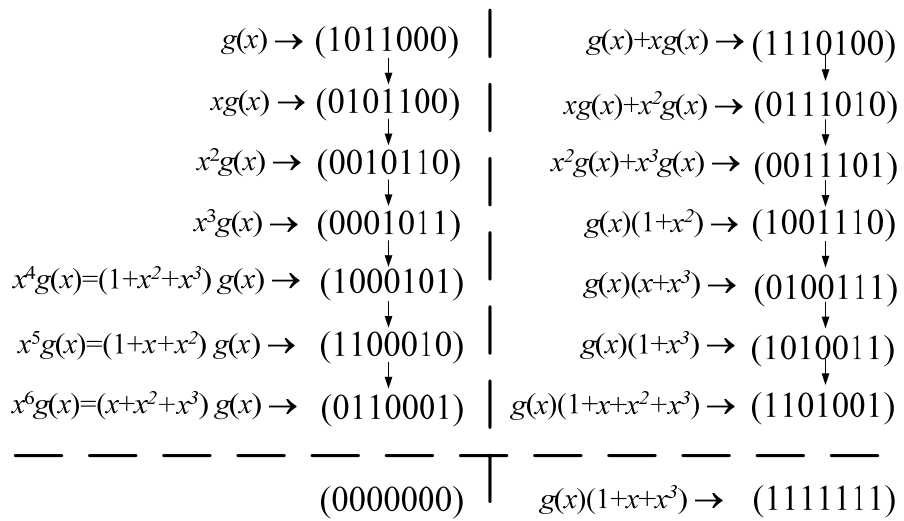
Prva kodna reč ovog cikličnog koda se može dobiti očitavanjem koeficijenata generišućeg polinoma a još tri linearno nezavisne kodne reči cikličnim pomeranjem ove reči:

$$\begin{aligned} g(x) &= (1011000) \\ xg(x) &= (0101100) \\ x^2g(x) &= (0010110) \\ x^3g(x) &= (0001011) \end{aligned}$$

pa ovom kodu odgovara generišuća matrica

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}.$$

Ovo su linearno nezavisni polinomi (vektori) koda (ideala). Ostale kodne reči su njihove linearne kombinacije. Na slici 6.5 prikazane su sve kodne reči ovog cikličnog koda i prikazan je način njihovog formiranja preko polinoma koji odgovaraju vrstama generišuće matrice. Jasno je da reči čija Hemingova težina iznosi $d=3$, postoji sedam reči koje čine “ciklični skup”, jer upravo toliko postoji mogućih cikličnih pomeraja reči koja odgovara polinomu $g(x)$. Sve ove kodne reči su linearno nezavisne, a zbirom dve reči iz ove grupe može se dobiti kodna reč Hemingove težine $d=4$. Njenim cikličnim pomeranjem može se formirati još jedna ciklična grupa iste veličine kao i prethodna. Kod sadrži i kombinacije svih nula i svih jedinica pa je dobijeni ciklični kod je ekvivalentan Hemingovom kodu (7,4), jer oba imaju isti spektar. Odavde se jasno vidi da se cikličnim pomerajem jedne reči ne mogu dobiti sve ostale reči iz ovog koda. Ipak, cikličnim pomerajem reči uvek se dobijaju reči koje takođe pripadaju cikličnom kodu, što i jeste u skladu s njegovom definicijom.



Slika 6.5 Ilustracija cikličnosti koda (7,4) opisanog u tekstu

Kontrolni polinom je

$$h(x) = (1+x)(1+x+x^3) = 1+x^2+x^3+x^4.$$

U ovome slučaju je dobijen množenjem preostalih faktora, ali je isto tako mogao biti dobijen i deljenjem, tj. $h(x) = (x^7+1) : g(x)$. I $h(x)$ obrazuje svoj kod (ideal):

$$\begin{aligned} h(x) &= (1011100) \\ xh(x) &= (0101110) \\ x^2 h(x) &= (0010111). \end{aligned}$$

Ako je proizvod ova dva polinoma ravan nuli, tj. ako je

$$g(x) \cdot h(x) = x^n - 1 = 0 \pmod{(x^n - 1)},$$

tada su odgovarajući vektori koeficijenata ortogonalni, pri čemu elemente jednog od vektora – bilo kojeg, zbog komutativnosti – treba uzeti u obrnutom redosledu. U ovom primeru je kontrolna matrica

$$\mathbf{H} = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

Lako je proveriti da vredi

$$\mathbf{GH}^T = \mathbf{0}.$$

Treba primetiti da su sve kolone u kontrolnoj matrici različite i da tek skupovi od po 3 kolone postaju linearno zavisni, što znači da je minimalna Hemingova težina nenulatih kodnih reči (a to znači i minimalno Hemingovo rastojanje) ravna 3. One takođe mogu biti predstavljene stepenima primitivnog elementa iz $GF(2^3)$. To polje se može dobiti bilo po modulu nesvodljivog polinoma x^3+x+1 , bilo po modulu x^3+x^2+1 :

	x^3+x+1 (primitivni element α)	x^3+x^2+1 (primitivni element $\beta=\alpha^3$)
(100)	$\alpha^0=1$	$\beta^0=1$
(010)	$\alpha^1=\alpha$	$\beta^1=\beta$
(001)	$\alpha^2=\alpha^2$	$\beta^2=\beta^2$
(110)	$\alpha^3=1 + \alpha$	$\beta^3=1 + \beta^2$
(011)	$\alpha^4=\alpha + \alpha^2$	$\beta^4=1 + \beta + \beta^2$
(111)	$\alpha^5=1 + \alpha + \alpha^2$	$\beta^5=1 + \beta$
(101)	$\alpha^6=1 + \alpha^2$	$\beta^6=\beta + \beta^2$
(100)	$\alpha^7(\alpha^0)=1$	$\beta^7(\beta^0)=1$

$$H = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} = [\alpha^0 \ \alpha^3 \ \alpha^5 \ \alpha^4 \ \alpha^6 \ \alpha^1 \ \alpha^2].$$

Odgovarajućim permutacijama može se dobiti matrica H' ekvivalentnog koda sa uređenim stepenima primitivnih elementa.

Činjenica da su u generišućoj i kontrolnoj matrici sledeće vrste samo ciklični pomeraji prethodnih vrsta ne predstavlja samo olakšicu u matematičkom opisu koda, već značajno doprinosi elegantnijoj, i samim tim ekonomičnijoj, konstrukciji koda i dekodera.

2) Potrebno je izvršiti kodovanje poruke $i=(1101)$ kodom opisanim u prethodnom primeru.

Proces kodovanja se može obaviti klasično, množenjem poruke generišućom matricom

$$c = [1 \ 1 \ 0 \ 1] \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1],$$

ili množenjem informacionog vektora $i(x) = x^3 + x + 1$ generišućim polinomom

$$\begin{aligned} c(x) &= m(x)g(x) = x^3(x^3 + x^2 + 1) + x(x^3 + x^2 + 1) + x^3 + x^2 + 1 \\ &= x^6 + x^5 + \cancel{x^3} + x^4 + \cancel{x^3} + x + x^3 + x^2 + 1 \\ &= x^6 + x^5 + x^4 + x^3 + x^2 + x + 1 \Leftrightarrow (1111111). \end{aligned}$$

Kodovanje putem množenja generišućim polinomom je znatno jednostavnije, kako pri hardverskoj tako i pri softverskoj realizaciji. Ovo će biti detaljnije objašnjeno u narednom odeljku.

3) Neka je dat polinom $x^{15}+1$ nad poljem $GF(2)$ i odgovarajući generišući polinom cikličnog koda $g(x)=x^4+x^3+1$ (svakako se može pisati i $g(x)=1+x^3+x^4$). Odgovarajući ciklični kod ima dimenzije $(15,11)$. Njega čine $2^{11}=2048$ kodnih reči. Pokazuje se da ovaj kod ima minimalno Hemingovo rastojanje $d=3$ (ekvivalentan je odgovarajućem Hemingovom kodu) i može da ispravi jednostruku grešku ili detektuje dvostruku grešku.

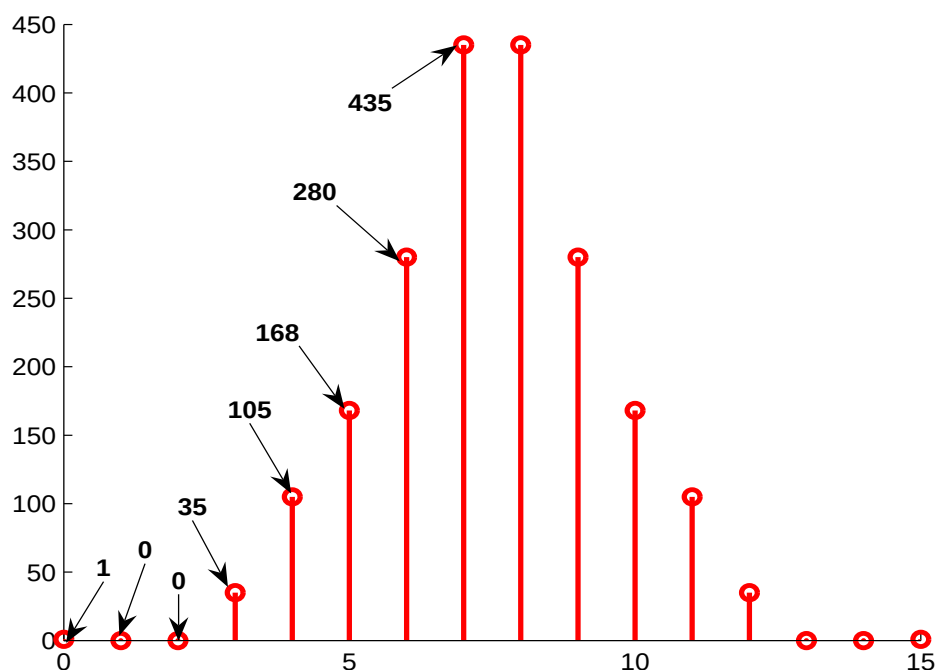
Spektar ovoga koda prikazan je na slici 6.6. Jasno se može primetiti da i u slučaju ovog koda, slično Hemingovim i Golejevim kodovima, postoji osa simetrije u odnosu na vrednost apscise 7,5. Štaviše, pokazuje se da je ovaj kod upravo ekvivalentan Hemingovom kodu $(15,11)$.

Ipak, ovde se može uočiti jedna naizgled zbunjujuća pojava. U prvom primeru ovog odeljka pokazano je da su ciklični pomeraji jedne kodne reči i sami kodne reči, što je u skladu sa definicijom cikličnog koda. Zato bi se moglo očekivati da moglo se očekivati da ukupan broj reči koje odgovaraju nekom Hemingovom rastojanju (jačina spektralne komponente) bude uvek deljiv sa dužinom kodne reči za $d \neq 0$ i $d \neq n$. Ovde to nije slučaj.

Primera radi, postoji ukupno 35 kodnih reči koje imaju Hemingovu težinu 3. Detaljnija analiza pokazuje da kod ovog koda postoji ukupno

- 15 cikličnih pomeraja reči (000000000010011)
- 15 cikličnih pomeraja reči (000000100000101)
- 5 cikličnih pomeraja reči (000010000100001).

Može se uočiti da kodna reč (000010000100001) ima samo pet različitih cikličnih pomeraja (šestim pomerajem dobija se reč identična polaznoj). Na taj način se jasno vidi da brojne vrednosti spektralnih komponenti cikličnog koda ne moraju biti deljive dužinom kodne reči.



Slika 6.6 Spektar posmatranog cikličnog koda $(15,11)$ □

Dakle, ciklični kod je, umesto generišućom matricom G , potpuno opisan generišućim polinomom $g(x)$ – tj. samo jednom vrstom generišuće matrice, jer su ostale vrste njeni ciklični pomeraji. Ekvivalentan zaključak vredi i za kontrolnu matricu H , odnosno polinom $h(x)$, s tim što umesto relacije $G \cdot H^T = 0$, vredi relacija $g(x) \cdot h(x) = x^n - 1 = 0 \pmod{(x^n - 1)}$. I ovde $g(x)$ i $h(x)$ daju dualne kodove. Prema tome, da bi se dobio ciklični kod, treba od informacionog polinoma $i(x)$, koji je stepena $k-1$, dobiti kodni polinom $v(x)$, stepena $n-1$, koji je deljiv generišućim polinomom $g(x)$. Na prijemu treba prispeli polinom $u(x)$ podeliti polinomom $g(x)$ i ako je ostatak ravan nuli, smatra se da nije bilo grešaka. Takođe se na prijemu $u(x)$ može pomnožiti sa $h(x)$ i proveriti da li je rezultat (ne ostatak!) ravan nuli (po modulu $x^n - 1$!).

Kao što je napred istaknuto, hardverska realizacija cikličnih kodova se zasniva na upotrebi linearnih prekidačkih kola kojima se može vršiti množenje i deljenje polinoma. U Dodatku B prikazano je kako se vrši ovo množenje i deljenje. Pri deljenju polinomom nekog stepena šema sadrži pomerački registar koji ima upravo toliko ćelija koliki je i stepen polinoma. U slučaju da se vrši množenje, šema sadrži pomerački registar s brojem ćelija koji je ravan stepenu polinoma kojim se vrši množenje. Ako se deli, pošto je $g(x)$ stepena $n-k$, to registar ima $n-k$ ćelija. U slučaju da se vrši množenje polinomom $h(x)$ stepena k i registar će imati k ćelija. Kod kratkih kodova, kada nema velike razlike u vrednosti $n-k$ i k , nije toliko važno da li vršiti množenje ili deljenje polinoma – međutim, kod dugačkih kodova, s velikom vrednošću k , daleko je ekonomičnije vršiti deljenje, jer tada registar ima samo $n-k$ ćelija. To je upravo razlog za primenu deljenja kod CRC postupka koji će biti opisan u daljem izlaganju.

Preostalo je još da se objasni kako se može generisati *sistematski ciklični kod*. Neka je polinom informacionih bita

$$i(x) = i_{k-1}x^{k-1} + \dots + i_1x + i_0$$

i neka je generišući polinom

$$g(x) = g_{n-k}x^{n-k} + \dots + g_1x + g_0.$$

Množenjem ova dva polinoma očigledno se dobija polinom

$$v(x) = i(x)g(x) = v_{n-1}x^{n-1} + \dots + v_1x + v_0,$$

koji mora biti kodni polinom, jer ima $g(x)$ kao faktor. Prema tome, šema za generisanje kodnih reči cikličnog koda (koder) može se realizovati koristeći množenje polinoma. Ovakvi koderi su krajnje jednostavni, ali im je mana što daju nesistematski kod – informacioni biti ne figurišu direktno u kodnoj reči (“ne vide se golim okom”). Naravno, na prijemu bi se oni mogli rekonstruisati deleći korektnu kodnu reč sa $g(x)$. Ovo bi moglo da unese kašnjenje u proces dekodovanja ako pri prenosu dođe do grešaka. Prvo bi trebalo izvršiti eventualnu korekciju grešaka, a onda dobijenu korektnu kodnu reč podeliti generišućim polinomom. Daleko je pogodnije ako bi kod bio sistematski. I to se može postići na relativno jednostavan način. Treba najpre informacioni polinom pomnožiti sa x^{n-k} , tako da se dobija nov informacioni polinom

$$i^*(x) = x^{n-k}i(x) = i_{k-1}x^{n-1} + \dots + i_1x^{n-k+1} + i_0x^{n-k}.$$

Sada ovaj rezultat treba podeliti generišućim polinomom $g(x)$ koji je stepena $n-k$. Očigledno da će ostatak biti stepena $n-k-1$, tj. da će imati ukupno $n-k$ koeficijenata,

$$r(x) = r_{n-k-1}x^{n-k-1} + \dots + r_1x + r_0.$$

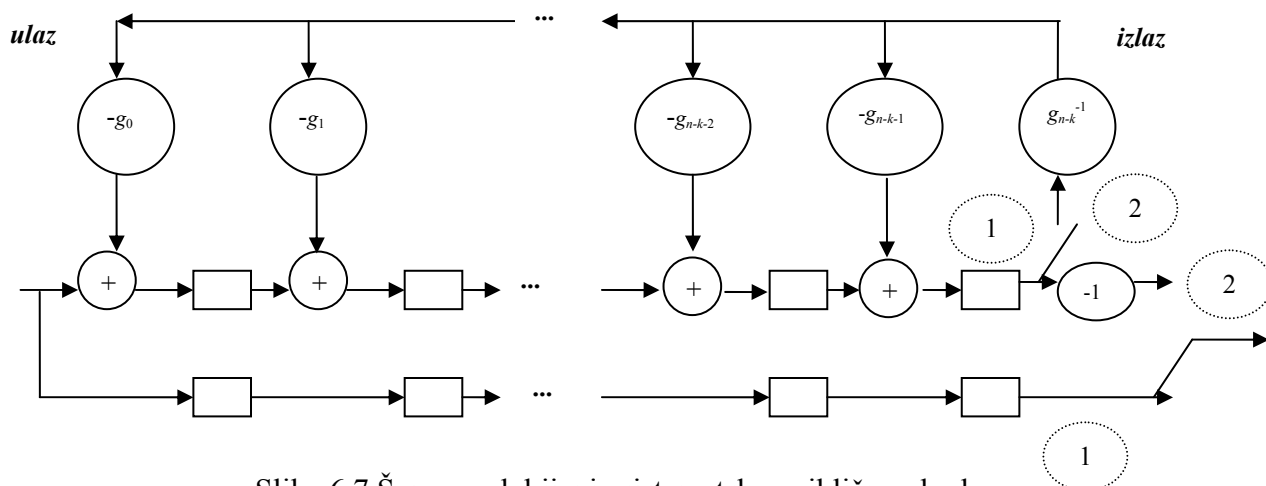
Kodna reč se sada dobija oduzimanjem ostatka $r(x)$ od polinoma $i^*(x)$, tj. kao

$$v(x) = x^{n-k}i(x) - r(x).$$

Ovome polinomu odgovara kodni vektor

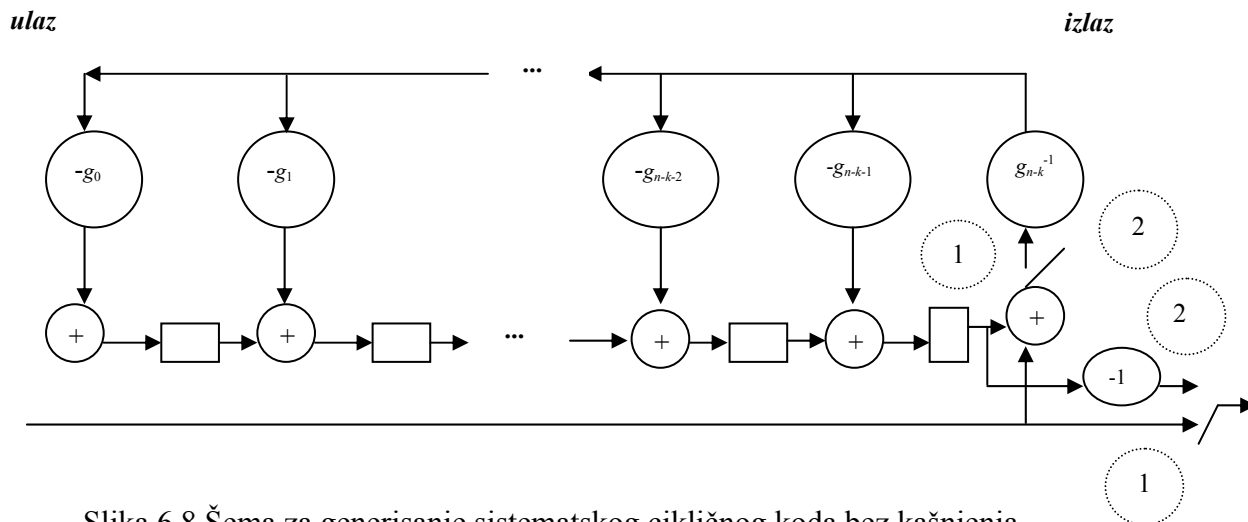
$$(i_{k-1}, \dots, i_1, i_0, r_{n-k-1}, \dots, r_1, r_0),$$

gde su informacioni biti neizmenjeni, a preostali biti su u stvari provere parnosti, čime je dobijen sistematski kod. Dakle, i ciklični kodovi se zasnivaju na dodavanju provera na parnost, ali na jedan daleko elegantniji način. Napred opisani postupak dobijanja sistematskog cikličnog koda može se ostvariti šemom na slici 6.7.



Slika 6.7 Šema za dobijanje sistematskog cikličnog koda

Na početku rada k informacionih bita ulazi u registar i istovremeno se šalje na liniju – prekidači su u položaju (1). Donja grana ima takođe $n-k$ ćelija što odgovara množenju sa x^{n-k} . Kada su informacioni biti otišli na liniju, prekidači se prebacuju u položaj (2) i na liniju se iščitava ostatak deljenja iz gornjeg registra pomnožen sa -1 (oduzimanje!). Istovremeno se na ulaz ne dovodi ništa – može se staviti prekidač ili se mogu dovoditi “nule”. Donji register, odnosno dodatno kašnjenje, može se izbeći sledećom šemom prikazanom na slici 6.8 koja daje ekvivalentan rezultat kao prethodna, ali bez kašnjenja.

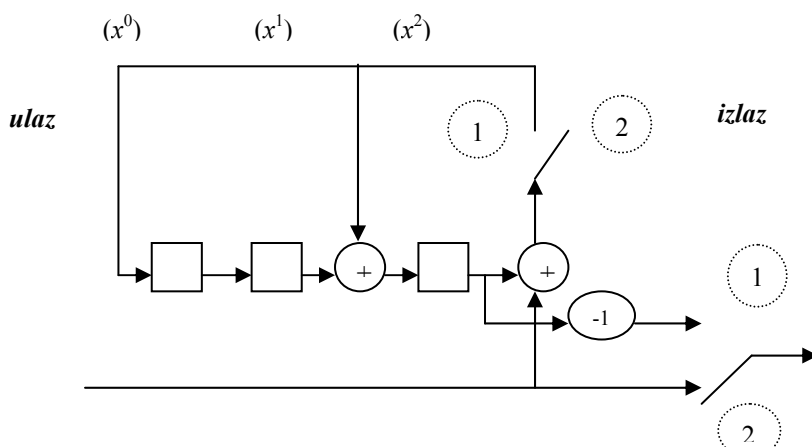


Slika 6.8 Šema za generisanje sistematskog cikličnog koda bez kašnjenja

Na početku su prekidači u položaju (1) i informacioni biti idu direktno na liniju, a u registru se formira ostatak. U toku rada sadržaji registra neće biti identični sadržajima registra u prethodnoj šemi, ali će po odlasku poslednjeg informacionog bita na liniju sadržaji oba registra biti identični. Sada se prekidači prebacuju u položaj (2) i opet iščitava ostatak pomnožen sa -1 .

Primeri:

1) Za slučaj $g(x) = x^3 + x^2 + 1$, dobija se, prema poslednjoj opštoj šemi, konfiguracija kao na slici 6.9.



Slika 6.9 Sistematski ciklični koder za generišući polinom $g(x) = x^3 + x^2 + 1$

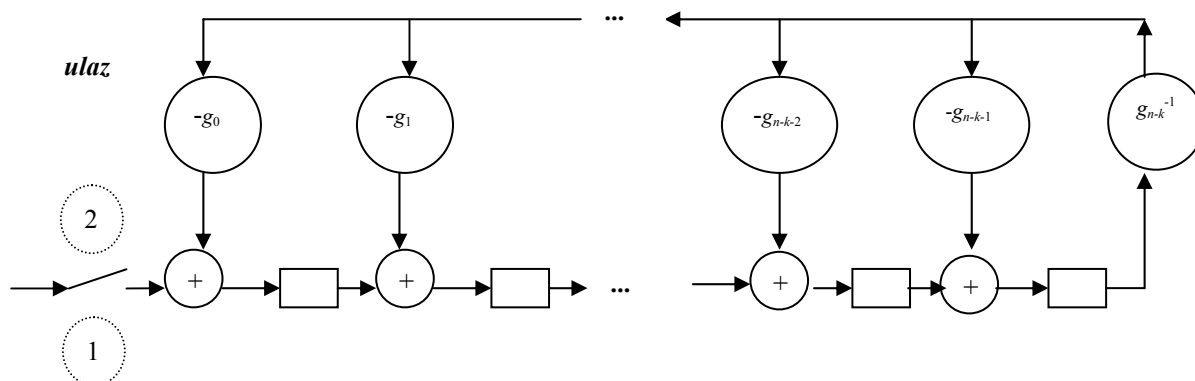
2) Neka je sistematski cikličan kod generisan na osnovu polinoma $g(x) = x^3 + x^2 + 1$. Odgovarajuće kodne reči i njihov polinomijalni zapis prikazani su u tabeli 6.1 (korišćene su oznake kao na slici 6.5). Informacioni polinomi su prvo pomnoženi sa x^3 (jer je $n-k=3$), zatim podeljeni sa $g(x)$ i određen je odgovarajući ostatak koji je smešten na prva tri bita kodne reči. Zato je informacioni deo smešten na kraju svake reči. Kako je minimalno Hemingovo rastojanje u ovom kodu $d=3$, a spektar identičan onome prikazanom na slici 6.1a, jasno je da je ovaj kod ekvivalentan Hemingovom kodu (7,4).

Tabela 6.1 Sistematski ciklični kod ekvivalentan Hemingovom kodu (7,4)

Poruke	Kodni vektori	Kodni polinomi
(0000)	0000000	$0 = 0 \cdot g(x)$
(1000)	1101000	$1+x+x^3 = 1 \cdot g(x)$
(0100)	0110100	$x+x^2+x^4 = x \cdot g(x)$
(1100)	1011100	$1+x^2+x^3+x^4 = (1+x) \cdot g(x)$
(0010)	1110010	$1+x+x^2+x^5 = (1+x^2) \cdot g(x)$
(1010)	0011010	$x^2+x^3+x^5 = x^2 \cdot g(x)$
(0110)	1000110	$1+x^4+x^5 = (1+x+x^2) \cdot g(x)$
(1110)	0101110	$x+x^3+x^4+x^5 = (x+x^2) \cdot g(x)$
(0001)	1010001	$1+x^2+x^6 = (1+x+x^3) \cdot g(x)$
(1001)	0111001	$x+x^2+x^3+x^6 = (x+x^3) \cdot g(x)$
(0101)	1100101	$1+x+x^4+x^6 = (1+x^3) \cdot g(x)$
(1101)	0001101	$x^3+x^4+x^6 = x^3 \cdot g(x)$
(0011)	0100011	$x+x^5+x^6 = (x+x^2+x^3) \cdot g(x)$
(1011)	1001011	$1+x^3+x^5+x^6 = (1+x+x^2+x^3) \cdot g(x)$
(0111)	0010111	$x^2+x^4+x^5+x^6 = (x^2+x^3) \cdot g(x)$
(1111)	1111111	$1+x+x^2+x^3+x^4+x^5+x^6 = (1+x^2+x^5) \cdot g(x) \quad \square$

$$\begin{array}{r}
 (x^2+1)x^3 \\
 (x^3+x+1) \\
 \hline
 = x^2
 \end{array}$$

Na prijemu treba proveriti da li je primljeni polinom (vektor) kodni ili nije. Jedan od načina je da se proveri da li deljiv sa $g(x)$. Ako je ostatak pri deljenju ravan nuli, tada se smatra da nije bilo grešaka. Za ovu proveru može se koristiti šema za deljenje prikazana na slici 6.10.



Slika 6.10 Šema dekodera cikličnog koda čiji je generišući polinom $g(x)$

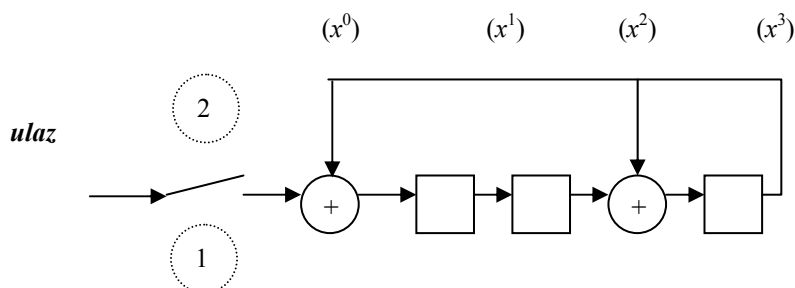
Primljeni vektor se uvodi u kolo za deljenje. Posle n koraka pregleda se registar gde se sada nalazi ostatak od deljenja sa $g(x)$. To je u stvari sindrom koji se sada može shvatiti i kao polinom $s(x)$. Ako je ovaj ostatak ravan nuli, tada se smatra da nije došlo do grešaka pri prenosu. Treba imati u vidu da se ovakvim deljenjem sadržaj primljenog vektora uništava, te da ga treba čuvati u još jednom baferu, da bi se mogli izdvojiti informacioni simboli. Ako sindrom nije jednak nuli, a smatra se da se dogodila samo jedna greška u kodnoj reči, tada se ovom šemom može odrediti i njena pozicija. Ovaj postupak će biti detaljnije izložen pri opisu Megitovog dekodera.

Primer:

Za napred razmatrani kod (7,4) s generišućim polinomom

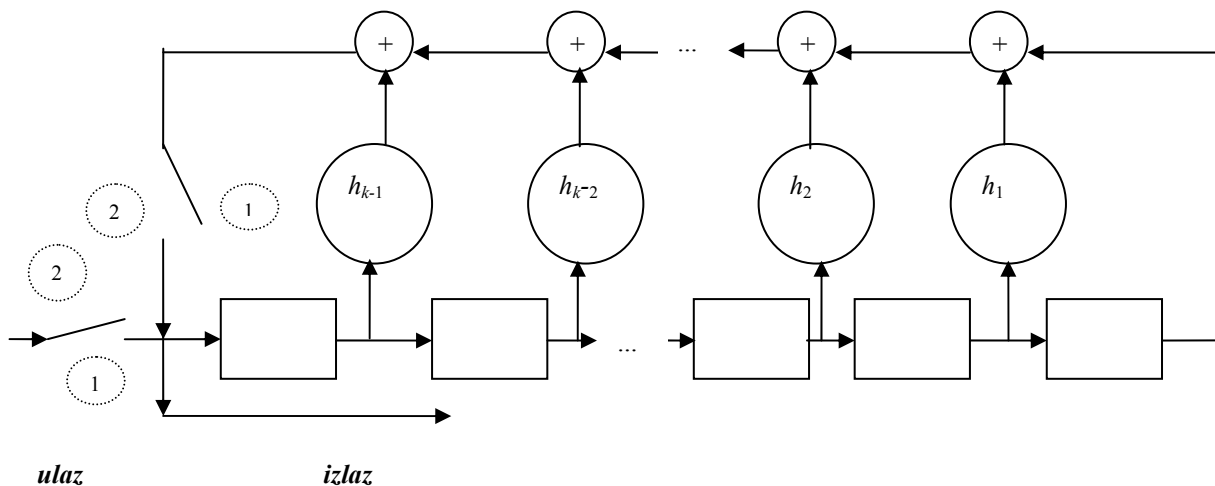
$$g(x) = x^3 + x^2 + 1$$

odgovarajuća šema dekodera data je na slici 6.11.



Slika 6.11 Dekoder cikličnog koda s generišućim polinomom $g(x) = x^3 + x^2 + 1$ □

U prethodnom izlaganju korišćen je polinom $g(x)$ i u koderu i u dekoderu. U slučaju da je polinom $h(x)$ nižeg stepena može se primeniti konfiguracija prikazana na slici 6.12. Ona se bazira na šemi za računanje rekurentnih relacija opisanoj u Dodatku B. Jedina je razlika što se ovde informacioni biti (kojih ima k) uvode s “leve strane” u registar dužine k . Oni istovremeno odlaze i na liniju, tj. radi se bez kašnjenja. Za to vreme su prekidači u položaju (1). Kada poslednji informacioni bit bude uveden u koder prekidači se prebacuju u položaj (2). Sada na liniju, opet bez kašnjenja, odlaze kontrolni biti koji se upravo formiraju na osnovu sadržaja registra.



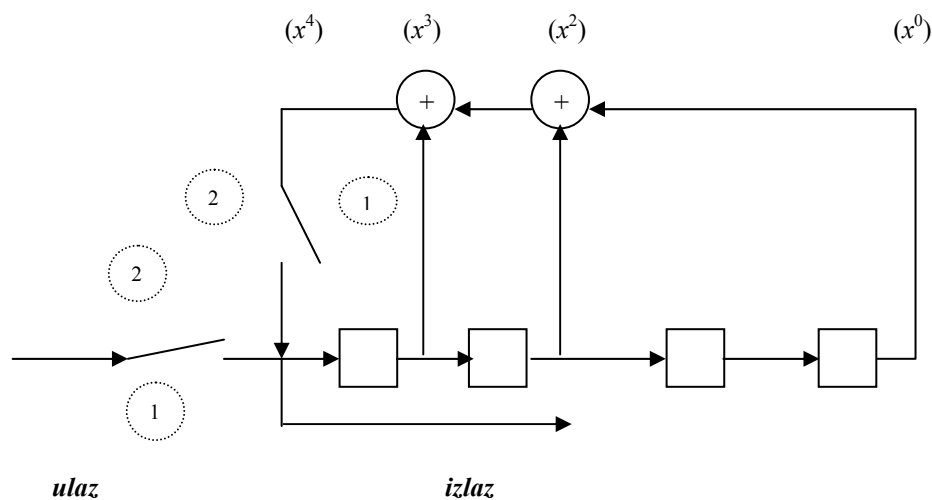
Slika 6.12 Koder cikličnog koda baziran na kontrolnom polinomu $h(x)$

Primer:

Za prethodno posmatrani kod (7,4) kontrolni polinom je

$$h(x)=x^4+x^3+x^2+1,$$

dok je odgovarajuća konfiguracija prikazana na slici 6.13.



Slika 6.13 Koder cikličnog koda s kontrolnim polinomom $h(x)=x^4+x^3+x^2+1$ □

6.2.1 Megitova teorema i Megitov dekodler

Megitova (*Meggitt*) teorema jednostavno povezuje sindrom – $s(x)$ – primljenog polinoma (kodne reči) $u(x)$ i ciklični pomeraj sindroma – $xs(x)$ – sa cikličnim pomerajem primljene reči $xu(x)$ na sledeći način, gde $R_{(.)}$ označava ostatak pri deljenju sa $(.)$ [4]:

Neka je $x^n - 1 = g(x)h(x)$, i $R_{g(x)}[u(x)] = s(x)$, tada je

$$R_{g(x)}[xu(x) \pmod{x^n - 1}] = R_{g(x)}[xs(x)].$$

Prema tome, **polinomi greške** $e(x)$, koji su ekvivalentni ranije korišćenim pojmovima vektora greške ($\mathbf{e}$), i njihovi sindromi zadovoljavaju sledeće

$$R_{g(x)}[xe(x) \pmod{x^n - 1}] = R_{g(x)}[xs(x)].$$

Dalje, ako je $e(x)$ polinom koji se može korigovati datim kodom sa sindromom $s(x)$, njegov ciklični pomeraj

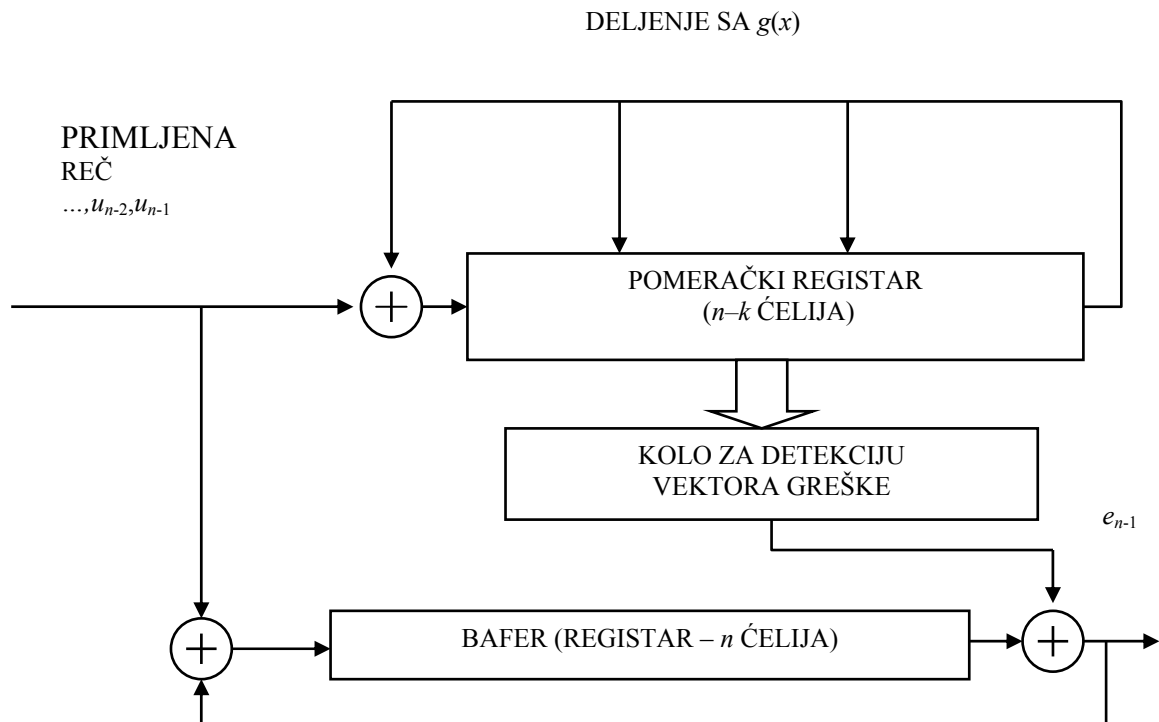
$$e'(x) = xe(x) \pmod{x^n - 1}$$

se takođe može korigovati koristeći sindrom

$$s'(x) = R_{g(x)}[e'(x)] = R_{g(x)}[xs(x)].$$

Poenta je da ako se znaju sindrom (polinom) i odgovarajući polinom greške, poznati su i sindromi za svaki ciklični pomeraj polinoma greške. Jedina operacija koja se mora vršiti je deljenje sindroma sa $g(x)$. Megitov dekodler upravo to i čini.

Generalno, korekcija grešaka se može sprovoditi računanjem sindroma – deljenjem dolaznog polinoma generišućim polinomom – i korišćenjem relacije 1:1 između sindroma i polinoma greške. Može se upravo koristiti i tabela. Ako se koristi Megitova teorema, veličina tabele je smanjena.



Slika 6.14 Konfiguracija Megitovog dekodera

Osnovna ideja Megitovog dekodera prikazana je na slici 6.14. Primljena reč ulazi u bafer i istovremeno se deli generišućim polinomom $g(x)$. Kada poslednji primljeni bit uđe u bafer, u gornjem registru je istovremeno formiran ostatak (sindrom). Po ulasku poslednjeg bita s linije u dekodera, ulaz se raskida, ili se na njega dovodi odgovarajući broj nula. Ovaj sindrom se poredi s test sindromima u kolu za detekciju vektora greške. Ako je utvrđeno slaganje, tada se najstariji bit, koji je upravo na izlazu bafera, koriguje ($e_{n-1}=1$). Zatim se u oba registra vrši pomeranje sadržaja za jedan korak. Na taj način je sindrom pomnožen sa x i podeljen sa $g(x)$, novi sadržaj registra je sada $R_{g(x)}[xs(x)]$, što je sindrom polinoma $xe(x)$ (mod x^n-1). Ako se nakon pretrage ovaj sindrom nađe i u kolu za detekciju, tada se desila greška na sledećem bitu ($e_{n-2}=1$) koji je upravo na izlazu bafera. Ovaj postupak se ponavlja n puta. Najniža povratna veza nije ni potrebna. Međutim, u nekim slučajevima, korigovana kodna reč u baferu može biti potrebna za neku dalju obradu.

Primer:

Neka je dat kod (15,11) (ekvivalentan Hemingovom kodu) koji koriguje jednu grešku u kodnoj reči. Generišući polinom je

$$g(x) = x^4 + x + 1$$

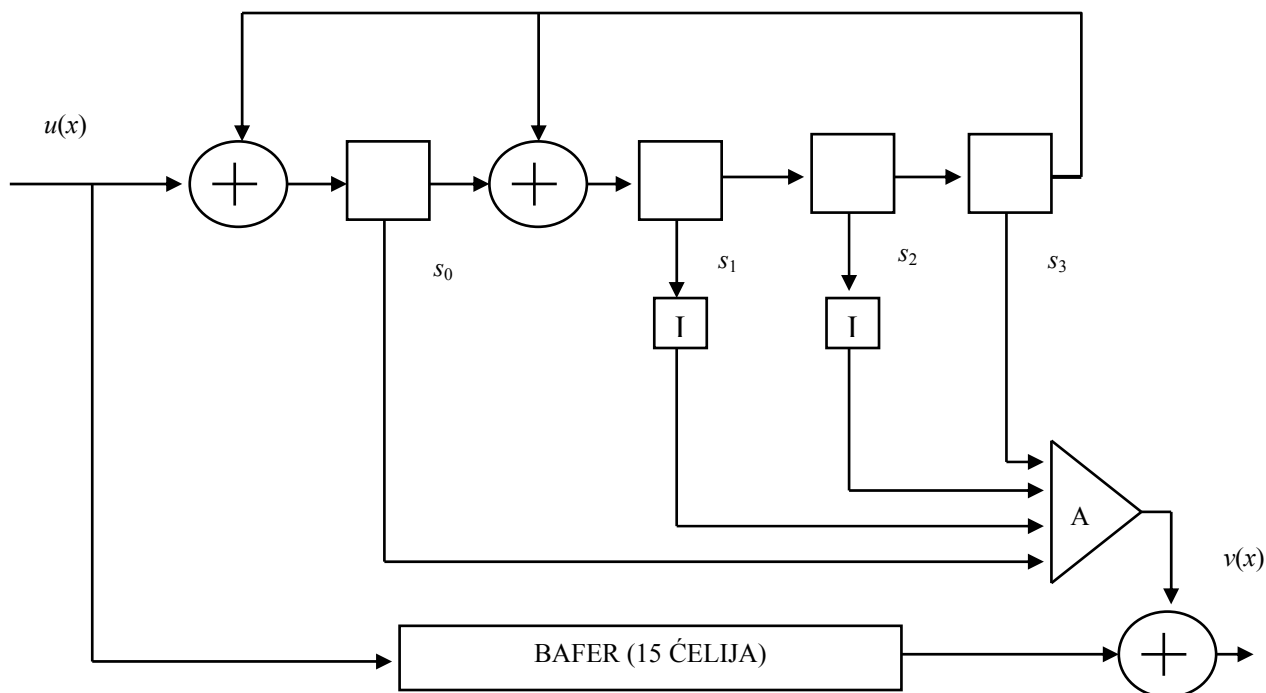
(kasnije će se videti da je to BCH kod koji ima kao korene generišućeg polinoma elemente α i α^2). Pošto se očekuju samo jednostruke greške, polinom greške koji odgovara pogrešci na najstarijoj poziciji je

$$e(x) = x^{14},$$

dok je odgovarajući sindrom

$$s(x) = x^3 + 1 \text{ (tj. vektor 1001)}.$$

Odgovarajući Megitov dekodera je prikazan na slici 6.15 – donja povratna sprega nije naznačena. Invertori su označeni sa I i samo se vektor 1001 traži putem i -kola (logička operacija AND) označenog sa A.



Slika 6.15 Megitov dekodera za ciklični kod čiji je generišući polinom $g(x) = x^4 + x + 1$

Ako je posle 15 prvih pomeraja (kada svi primljeni biti uđu u bafer) vrednost sindroma 1001, tada se koriguje u_{14} . Ako to nije slučaj, vrši se sledeće pomeranje sadržaja u registrima. Ako je novi sindrom 1001, tada je $e(x) = x^{13} \Rightarrow e_{13}=1$ i bit u_{13} se koriguje. Na taj način bilo koja greška na samo jednom bitu može se ispraviti. Naravno, kada se greška ispravi, nema daljih ispravljanja, ali se u registrima i dalje vrši pomeranje do kraja dok se ne dobije kompletna korektna reč na izlazu. Dakle, potrebno je ukupno $15+15=30$ pomeraja da bi se izvršio kompletan proces ispravljanja grešaka. $\square$

Postoji više različitih oblika Megitovog dekodera, ali je originalna ideja uvek ista, kao što je napred izložena. Ovo nije algebarski dekođer. Nema računanja u polju Galoa, izuzev deljenja sa $g(x)$. Međutim, kompleksnost Megitovog dekodera raste veoma brzo ako se povećava broj grešaka koje se žele korigovati. Generalno, on se ne koristi za korekciju više od 3 pojedinačne greške ili jednog paketa grešaka. U poslednjem slučaju, odgovarajući polinom greške ima oblik

$$e(x) = x^k + x^{k-1} + \dots + x^{k-b+1},$$

gde je dužina paketa b . Treba imati u vidu da ne moraju svi koeficijenti ovoga polinoma (izuzev prvog i poslednjeg) biti različiti od nule.

6.2.2 Provera ciklične redundanse (CRC postupak)

Postupak provere ciklične redundanse (CRC od “*Cyclic Redundancy Check*”) zasniva se na formiranju sistematskog cikličnog koda, opisanog na početku odeljka 6.3. Kod ovog postupka biti koji odgovaraju korisnim podacima prenošenim u jednom transmisionom ramu predstavljaju informacione bite koji se koduju cikličnim kodom na osnovu pravila

$$c(x) = i(x)x^{n-k} + \text{rem} \left\{ \frac{i(x)x^{n-k}}{g(x)} \right\} = i(x)x^{n-k} + r(x),$$

gde je sa $\text{rem}\{\cdot\}$ označen ostatak pri deljenju polinoma. Na ovaj način su informacioni i kontrolni biti u kodnoj reči jasno razdvojeni, jer je u pitanju sistematski ciklični kod. Zato se može smatrati da je na podatke koje treba preneti “prikačen” CRC dodatak koji je obično znatno manjih dimenzija od ostatka paketa ($n-k \ll k$). Na ovaj način je, uz moguće dodavanje bita koji čine zaglavlje (u daljoj analizi se zanemaruje), kompletno formiran transmisioni ram.

Pri prenosu kroz kanal, na bite koji odgovaraju ovom ramu vrši se superponiranje grešaka opisanih polinomom greške $e(x)$. Na prijemu se vrši deljenje polinoma koji odgovara primljenoj sekvenci generišućim polinomom. Ostatak pri ovom deljenju na strani dekodera sada ima oblik

$$r'(x) = \text{rem} \left\{ \frac{c(x) + e(x)}{g(x)} \right\}.$$

Pošto biti koji čine transmisioni ram odgovaraju jednoj kodnoj reči cikličnog koda, ako nije bilo grešaka pri prenosu ovaj ostatak će biti ravan nuli. U slučaju da je ostatak različit od nule, poznato je da se desila greška pri prenosu ali ne i na kojoj poziciji. Zato CRC može da detektuje greške, ali ne može da ih koriguje. Ipak, on ima ogroman značaj ako se kombinuje sa tehnikama zaštitnog kodovanja. Tada se na ovaj način omogućava provera da li je prenošeni niz informacionih bita (grupisanih u ram ili paket) nakon dekodovanja ispravno primljen. O ovome će više biti reči u odeljku 7.3.

Pošto dužina “kodne reči” – rama – može da bude promenljiva, to se mora pribeći skraćivanju cikličnog koda, tako što se smatra da su nedostajući najstariji informacioni simboli ravni nuli, pa se oni i ne šalju, a pošto imaju vrednost nula ne utiču na ostatak pri deljenju. Generišući polinom koda je u praksi obično relativno niskog stepena ($n-k \leq 32$) dok dužina kodne reči n može da bude i više stotina bita, pa k može biti izuzetno veliko. Zato se pri kodovanju primenjuje deljenje polinomom $g(x)$ za formiranje sistematskog koda, a proveru na prijemu se takođe ostvaruje deljenjem istim polinomom.

Izbor polinoma za CRC je bio predmet više istraživanja [10,11,12]. Galager [13] primećuje da je uobičajena praksa da se za generišući polinom izabere proizvod jednog nesvodljivog polinoma, tj. polinoma koji se više u datom polju ne može faktorizovati, i polinoma $x+1$ koji osigurava detekciju svih vektora greške s neparnim brojem grešaka. Greška je nedetektabilna ako je $e(x)$ deljiv sa $g(x)$. U opštem slučaju, može se detektovati sledeće:

- sve jednostruke greške ako $g(x)$ ima više nego jedan nenulti član;
- sve dvostruke greške ako $g(x)$ ima faktor sa tri člana;
- bilo koji neparan broj grešaka, ako $p(x)$ sadrži faktor $x+1$;
- svaki paket grešaka dužine manje ili jednake od $n-k$;
- deo paketa grešaka dužine $n-k+1$ – njih $1-2^{-(n-k-1)}$;
- deo paketa grešaka dužine veće od $n-k+1$ – njih $1-2^{-(n-k)}$.

U praksi obično koriste kodovi 16. ili 32. reda, dok su dužine informacionih blokova reda nekoliko stotina pa i hiljada bita. Najčešće korišćeni generišući polinomi CRC kodova preporučenih od strane ITU-T prikazani su u tabeli 6.2.

Tabela 6.2 Generišući polinomi CRC kodova

Sistem	Generišući polinom
IEEE 802.16 (WiMax)	$g_{CRC-8}(x) = x^8 + x^2 + 1 =$ $= (x+1)(x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + 1)$
ATM (zaglavlje)	$g_{CRC-8}(x) = x^8 + x^2 + x + 1$
X25, V41, CDMA, Bluetooth, HDLC, PPP	$g_{CRC-16}(x) = x^{16} + x^{12} + x^5 + 1 =$ $= (x+1)(x^{15} + x^{14} + x^{13} + x^{12} + x^4 + x^3 + x^2 + x + 1)$
IBM CRC-16, ANSI, USB, SDLC	$g_{CRC-16}(x) = x^{16} + x^{15} + x^2 + 1$ $= (x+1)(x^{15} + x + 1)$
UMTS, WLAN.	$g_{CRC-24}(x) = x^{24} + x^{23} + x^6 + x^5 + x + 1$
ETHERNET	$g_{CRC-32}(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} +$ $+ x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$

Performanse CRC koda se ocenjuju na osnovu procenta vektora greške koji se mogu otkriti. Treba zapaziti da se superponiranjem sekvence grešaka sa poslatom kodnom reči na prijemu može primiti opet jedna od kodnih reči. U tom slučaju dekodirer će smatrati da nije bilo grešaka pri prenosu, jer je na njegov ulaz pristigla sekvenca bita koja odgovara polinomu koji je deljiv odgovarajućim generišućim polinomom. Ako je kodna reč dužine n bita, na ulaz dekodera može doći ukupno 2^n mogućih binarnih kombinacija, od čega je samo 2^k validnih kodnih reči. Ovo dalje znači da se sa porastom faktora $n-k$ povećava sposobnost koda da detektuje greške, što se lako uočava na osnovu prethodne analize, a konkretna verovatnoća nedetektovane greške zavisi od statistike grešaka u kanalu.

Pošto se CRC postupak najviše koristi pri prenosu fajlova po kanalima gde se često javljaju paketi grešaka – na primer, telefonski kanal – to pretpostavka o statističkoj nezavisnosti grešaka nije opravdana, pa nije toliko ni bitno Hemingovo rastojanje. Jedan CRC kod sa $n-k$ kontrolnih bita može da detektuje $(1-2^{n-k}) \cdot 100\%$ svih vektora greške [11]. Takođe se često vrše izvesne modifikacije CRC postupka, da bi se izbegla zamena CRC polja s drugim poljima (recimo flegom), ili dugi niz simbola istoga znaka, što je važno kod zapisa. Ove modifikacije se obično svode na komplementiranje nekih ili svih bita CRC-a. Naravno da one ne utiču na performanse ovoga postupka.

Primeri:

1) Binarna poruka $i=(01100111)$ prenosi se kroz sistem koji koristi cikličnu proveru redundanse sa generišućim polinomom $g(x) = x^4 + x^3 + 1$. Potrebno je odrediti kodnu reč koja odgovara poslatoj poruci.

Kako je dužina informacione sekvence $k=8$ a najviši stepen u generišućem polinomu $n-k=4$, u pitanju je CRC kod (12,8). Pošto je informacioni polinom $i(x) = x + x^2 + x^5 + x^6 + x^7$, kodna reč sistematskog koda dobija se u dva koraka.

$$i(x)x^4 = x^5 + x^6 + x^9 + x^{10} + x^{11}$$

$$r(x) = \text{rem} \left\{ \frac{i(x)x^4}{x^4 + x^3 + 1} \right\} = x + x^2$$

pa je $c=(0110011011)$, gde je informaciona reč sadržana u kodnoj počev od pete pozicije.

Neka se pri prenosu kroz kanal desila greška na prvoj i trećoj poziciji ove kodne reči. Tada vektor greške i primljeni vektor imaju oblik

$$e=(00000000101), \quad c'=(11110110111).$$

Proces dekodovanja sada se svodi na računanje ostatka pri deljenju polinoma koji odgovara primljenom vektoru generišućim polinomom. U ovom slučaju dobija se ostatak

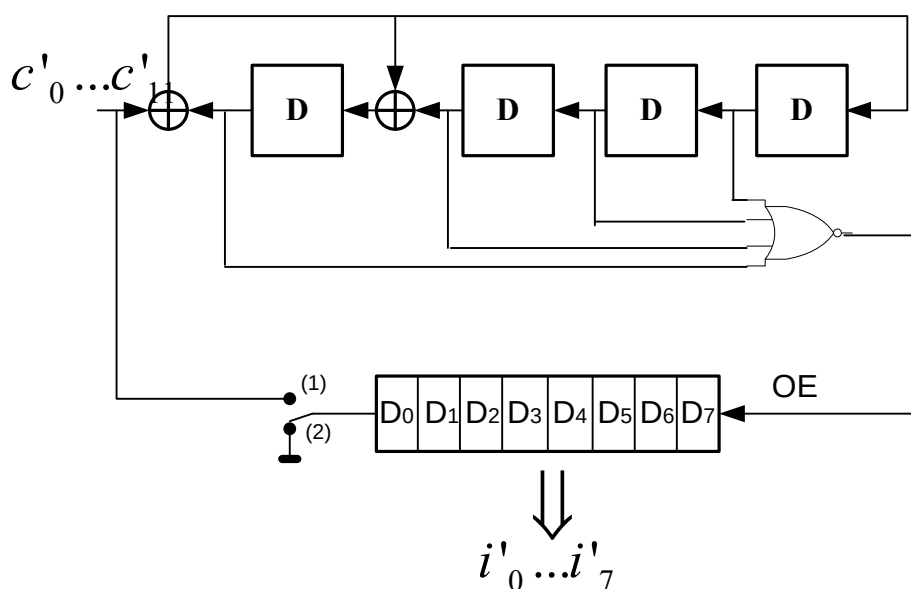
$$r'(x) = \text{rem} \left\{ \frac{1 + x + x^2 + x^3 + x^5 + x^6 + x^9 + x^{10} + x^{11}}{1 + x^3 + x^4} \right\} = \text{rem} \left\{ \frac{c'(x)}{g(x)} \right\} = x^3 + 1$$

Kako ovom ostatku odgovara sindrom $S=[1001] \neq 0$, detektovano je da pri prenosu ima grešaka ali ne koliko i na kojoj poziciji se nalaze.

Čitaocu se prepušta da proveriti da li je bi u slučaju da nije bilo greške pri prenosu sindrom bio ravan nuli.

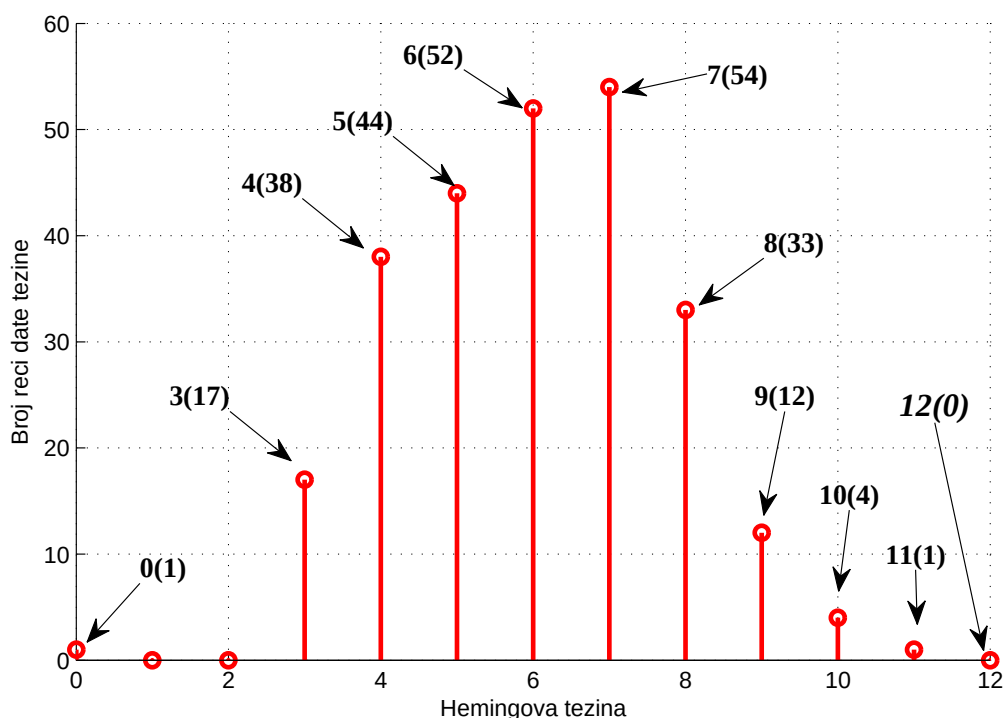
Formiranje kodne reči CRC koda obavlja se po principu ranije prikazanom na slici 6.10. Nešto interesantnija je hardverska realizacija CRC dekodera, prikazana na slici 6.16, jer ona bolje opisuje specifičnosti CRC postupka.

Neka na ulaz dekodera stižu biti poslate kodne reči na koje je superponirana sekvenca grešaka. Kada je prekidač u položaju (1) prvih $k=8$ (informacionih) bita se serijski upisuje u donji pomerački registar sa osam memorijskih ćelija i istovremeno stiže u kolo za deljenje. Nakon ovoga prekidač se prebacuje u položaj (2) čime se zabrani dalji upis podataka, a u kolu za deljenje se očitava ostatak pri deljenju. Ako je ostatak različit od nule signal ne dozvoljava se čitanje donjeg registra tako što se postavi vrednost signala OE=0 (OE od "Output Enabled") i zahteva se retransmisija podataka. U slučaju da je ostatak pri deljenju ravan nuli, smatra se da nije bilo grešaka pri prenosu pa se postavlja vrednost OE=1, sadržaj donjeg registra se paralelno pročita i informacioni biti se proslede korisniku.



Slika 6.16 Hardverska realizacija CRC dekodera (12,8)

Na slici 6.17 dat je spektar koji odgovara posmatranom CRC kodu (12,8). Za razliku od do sada prikazanih slučajeva, spektar ovog koda je nesimetričan. Osim toga, vidi se da u kodu ne postoji kodna reč koja sadrži sve jedinice (ne postoje reči s Hemingovom težinom $d=12$, što je na slici označeno kurzivom), tj. ne postoji neutralni element u odnosu na množenje (postojanje ovog elementa nije neophodno po definiciji linearnog blok koda, ali je neophodno za neskraćeni ciklični kod!). Oba efekta su posledica činjenice da je ovaj kod skraćen, tj. nastao je eliminisanjem tri bita iz informacione reči koda (15,11), prikazanog ranije na slici 6.7. Na ovaj način se i informaciona i kodna reč skraćuju za tri bita čime se delimično menjaju performanse polaznog koda ali se ne smanjuje njegovo minimalno Hemingovo rastojanje ni sposobnost ispravljanja grešaka.



Slika 6.17 Spektar CRC koda (12,8)

2) Primena CRC kodova je ušla u niz preporuka i standarda za digitalni prenos (i zapis) [14]. Na primer, u okviru HDLC (*High-Level Data-Link Control Protocol*) protokola, ram koji se koristi za prenos ima sledeći format [15]:

- početni fleg od 8 bita;
- adresno polje od 8 bita (koje se može proširivati u inkrementima od 8 bita);
- kontrolno polje (koje se može proširivati u inkrementima od 8 bita);
- informaciono polje proizvoljne veličine;
- polje za rezultat provera na parnost (*frame check field*) od 16 bita (opciono 32 bita);
- završni fleg od 8 bita.

Za izračunavanje provera na parnost koristi se skraćeni ciklični kod čiji je generišući polinom šesnaestog stepena

$$g(x) = x^{16} + x^{12} + x^5 + x^1 = (x^{15} + x^{14} + x^{13} + x^{12} + x^4 + x^3 + x^2 + x + 1) \cdot (x + 1).$$

Ovaj kod se obično obeležava sa CRC-CCITT (postoji i drugi polinom šesnaestog stepena – CRC-SDLC). Čitaocu se prepušta da, na osnovu opisa u prethodnom odeljku, skicira odgovarajući koder bez kašnjenja i dekode. Treba primetiti da se ostatak od deljenja na predaji može da formira kako ulazni biti stižu u koder – u “realnom vremenu” – a da se pri tome ulazni biti odmah prosleđuju dalje na liniju. Kada se završi deljenje, ostatak iz registra se iščitava u odgovarajuća polja rama. Na prijemu se sve takođe odigrava u “realnom vremenu”. Kako biti rama pristižu, u dekoderu se vrši deljenje i kada u dekoder uđu i 16 bita CRC-a, na osnovu stanja registra se konstatuje da li se desila greška ili ne. Kao što je već rečeno, ovom prilikom se “uništavaju” ulazni biti u dekoder, pa se primljeni biti moraju čuvati u posebnom registru. □